

Asignación de características musicales para ambientación comercial

Ing. Carlos Alberto Rivas Araque

Carrera de Especialización en Inteligencia Artificial

Director: Esp. Ing. Martín Moreyra (GatekeeperX)

Jurados:

Esp. Ing. Natanael Emil Ferrán (FIUBA)
Mg. Lic. Pablo Carlos Zizzutti (FIUBA)
Esp. Ing. Ana Laura Diedrichs (FIUBA)

Ciudad de Buenos Aires, junio de 2026

Índice general

Resumen	I
1. Introducción general	1
1.1. Extracción de información musical	1
1.2. Contexto de aplicación	1
1.3. Objetivos y alcance	2
1.3.1. Objetivo general	2
1.3.2. Objetivos específicos	2
1.3.3. Alcance	2
1.4. Estado del arte	3
1.4.1. Procesamiento digital de señales y extracción a bajo nivel	3
1.4.2. Modelos preentrenados y soluciones comerciales	3
1.4.3. Síntesis comparativa	4
2. Introducción específica	5
2.1. Fuentes de datos de canciones	5
2.2. Representaciones numéricas de la música	5
2.3. Información semántica del audio	7
2.4. Modelos de inteligencia artificial utilizados	8
2.5. Despliegue e integración	8
3. Diseño e implementación	11
3.1. Conjuntos de datos y extracción de atributos	11
3.1.1. Análisis comparativo de conjuntos de datos públicos	11
GTZAN	11
FMA estándar	12
Million Song Dataset (MSD)	13
AudioSet	13
3.1.2. Reducción de dimensionalidad mediante estadísticas	14
3.1.3. Definición del espacio de variables	16
3.2. Ejecución de tareas sobre el conjunto de datos	17
3.2.1. Distribución y tratamiento de las variables objetivo	17
Loudness	17
Key y mode	18
Tempo	19
Acousticness, danceability, energy, instrumentality y valence	20
Speechiness	21
Liveness	22
Top-5 primary mood	23
Top-5 secondary mood	24
Suggested genres	25

3.2.2.	Selección de variables de entrada	26
3.3.	Modelado y optimización	27
	Modelado mediante perceptrón multicapa (MLP)	27
	Modelado mediante Gradient Boosting (LightGBM)	28
	Optimización de hiperparámetros y regularización	28
3.4.	Arquitectura de despliegue e integración	29
3.4.1.	Diseño del servicio predictivo	30
3.4.2.	Flujo de integración	31
4.	Ensayos y resultados	33
4.1.	Resultados por modelo	33
4.1.1.	Acousticness, danceability, energy, instrumentalness y valence	33
4.1.2.	Loudness	35
4.1.3.	Key y mode	37
4.1.4.	Tempo	38
4.1.5.	Speechiness y liveness	40
4.1.6.	Suggested genres	42
4.1.7.	Primary y secondary moods	44
4.2.	Ejemplo de inferencia	46
5.	Conclusiones	49
5.1.	Conclusiones generales	49
5.2.	Próximos pasos	50
A.	Apéndice A: Respuesta JSON de variables predictivas por modelo	51
B.	Apéndice B: desglose armónico por tonalidad	53
C.	Apéndice C: respuesta JSON de inferencia	55
	Bibliografía	57

Índice de figuras

1.1. Interfaz de consulta en Tunebat	4
2.1. Representaciones visuales del audio	6
2.2. Probabilidades de detección semántica	8
3.1. Algoritmo de extracción de atributos	14
3.2. Proceso de reducción de dimensionalidad	15
3.3. Distribución de loudness	18
3.4. Distribución de key y mode	18
3.5. Distribución de tempo	19
3.6. Distribuciones de características perceptibles	20
3.7. Distribución de speechiness	21
3.8. Distribución de liveness	22
3.9. Distribución de primary mood	23
3.10. Distribución de frecuencias secondary mood	24
3.11. Distribución de frecuencias de suggested genres	25
3.12. Bloques del perceptrón multicapa para las variables perceptibles.	27
3.13. Resumen del modelado LightGBM.	28
3.14. Flujo de optimización de parámetros para los modelos LightGBM.	29
3.15. Diagrama de bloques de la aplicación predictiva.	30
3.16. Diagrama de despliegue e integración.	31
4.1. Curvas de aprendizaje del modelo MLP.	33
4.2. Relación de valores reales y estimados en el modelo MLP.	34
4.3. Top-15 variables predictivas del modelo MLP.	35
4.4. Curvas de aprendizaje para loudness.	35
4.5. Relación de valores reales y estimados para Loudness.	36
4.6. Top-15 variables predictivas para loudness.	36
4.7. Matriz de confusión de key y mode.	37
4.8. Top-15 variables predictivas para key y mode.	38
4.9. Relación de valores actuales y estimados para tempo.	39
4.10. Relaciones lineales y entre el tempo actual y el estimado con Essentia.	39
4.11. Histogramas de frecuencias del tempo actual y el estimado con Essentia.	39
4.12. Pruebas preliminares (izquierda) y actuales (derecha) para speechiness y liveness.	41
4.13. Top-15 variables predictivas para speechiness.	41
4.14. Top-15 variables predictivas para liveness.	42
4.15. Curvas de aprendizaje del modelo para la clasificación de géneros.	42
4.16. Matriz de confusión normalizada para la clasificación de 197 géneros musicales.	43
4.17. Top-15 variables predictivas para género.	43
4.18. Matriz de confusión para primary mood.	44

Índice de tablas

2.1. Atributos representativos	7
3.1. Desglose del espacio de variables	16
3.2. Distribución porcentual de speechiness antes y después del pre- procesamiento.	22
3.3. Distribución porcentual de liveness antes y después del preproce- samiento.	23
3.4. Resumen de parámetros usados para selección de variables.	26
3.5. Resumen de hiperparámetros óptimos por objetivo.	30
4.1. Comparativa de métricas de dos experimentos de la red neuronal MLP.	34
4.2. Métricas de Essentia y del modelo de key y mode.	38
4.3. Resultados de la estimación de tempo de Essentia.	40
4.4. Métricas en el modelado de speechiness y liveness.	40
A.1. Distribución del tipo de variables predictivas en los modelos.	51
B.1. Desglose porcentual de predicciones según su relación armónica para las 24 tonalidades.	53

Asimismo, la herramienta cuantifica algunas características en una escala porcentual de 0 a 100, mostrando para este caso específico niveles de acusticidad (77 %) e instrumentalidad (76 %), una baja probabilidad de presencia de habla (*speechiness* en 4 %) y público en vivo (*liveness* en 9 %), además de registrar el volumen medio en decibelios (-8 dB).

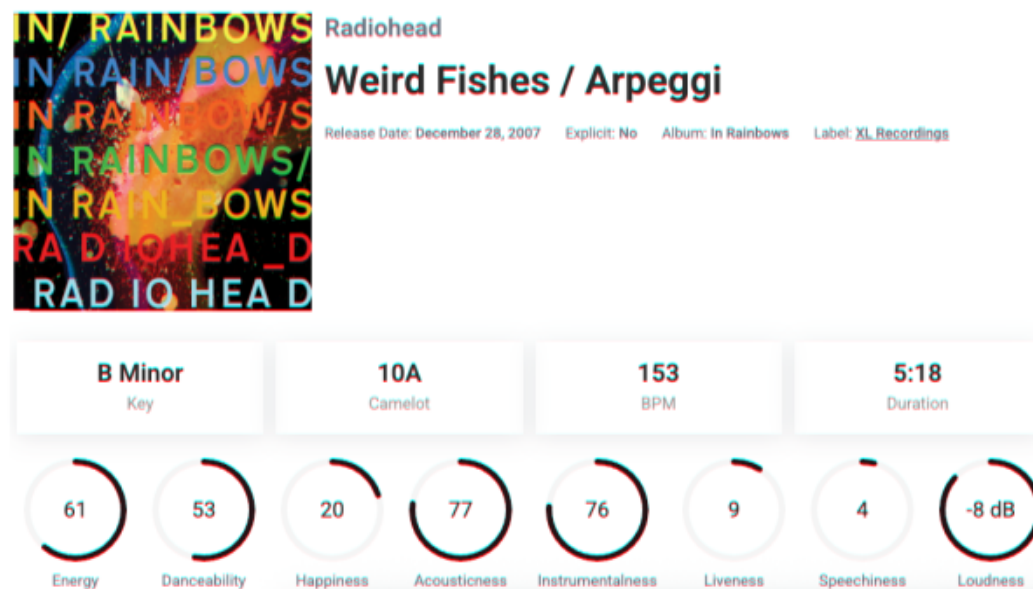


FIGURA 1.1. Ejemplo de visualización de características musicales en Tunebat¹.

1.4.3. Síntesis comparativa

Las tecnologías existentes ofrecen herramientas útiles para el objetivo del trabajo. Las bibliotecas de procesamiento digital de señales permiten extraer representaciones numéricas de las ondas sonoras, y los modelos preentrenados ofrecen representaciones semánticas. Por otro lado, las soluciones comerciales proveen servicios de etiquetado para canciones.

Sin embargo, persisten limitaciones operativas para su aplicación directa en el entorno de la empresa. Los servicios de terceros generan costos recurrentes, limitan la cantidad de consultas y crean dependencia tecnológica. Las plataformas web de uso manual no son viables para procesar catálogos con miles de canciones. Además, los modelos no coinciden con las características y las clases específicas que usa Plusyc Live.

Por estos motivos, resulta necesario desarrollar una solución propia. El propósito de este trabajo es aprovechar las técnicas actuales de inteligencia artificial para crear un sistema independiente. Esto permite analizar directamente los archivos de audio, etiquetarlos según las necesidades de la empresa y así reducir la dependencia de proveedores externos.

¹Imagen tomada de <https://tunebat.com/Info/Weird-Fishes-Arpeggi-Radiohead/4wa7m3n1axpde6rps/>
 Imagen tomada de GPs: <https://bit.ly/4nJ8RWg>

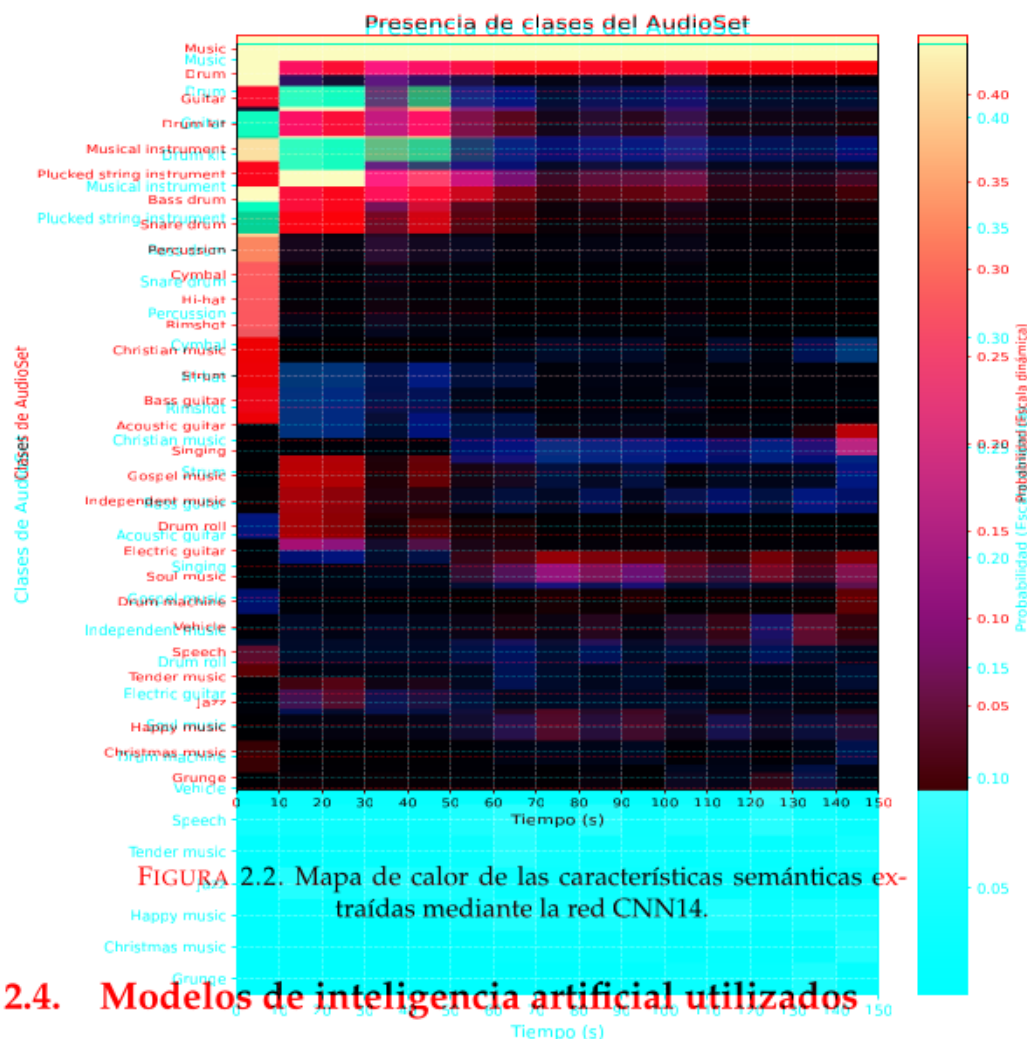


FIGURA 2.2. Mapa de calor de las características semánticas extraídas mediante la red CNN14.

2.4. Modelos de inteligencia artificial utilizados

Una vez obtenidos los descriptores numéricos y semánticos, es necesario aplicar algoritmos de predicción. Para este fin, se utilizaron dos enfoques principales:

- Algoritmos LightGBM: se integró la arquitectura LightGBM, basada en el ensamble de árboles de decisión. Esta herramienta destaca por su alta velocidad computacional y eficiencia en tareas generales de predicción [22]. En este caso también se usó Optuna para la búsqueda de parámetros óptimos [23].

Una vez obtenidos los descriptores numéricos y semánticos, es necesario aplicar algoritmos de predicción. Para este fin, se utilizaron dos enfoques principales:

- Redes MLP: los perceptrones multicapa son redes neuronales artificiales con capacidad para modelar relaciones complejas. LightGBM, entre otras aplicaciones, predecir múltiples variables continuas de forma simultánea [24].

2.5. Despliegue e integración

La integración del sistema predictivo requirió del despliegue de una interfaz de programación de aplicaciones. Para su construcción se utilizó Fast API [25] como marco de trabajo. Esta herramienta facilita el desarrollo de servicios web de alto rendimiento basados en Python [26]. El servicio fue contenedorizado con Docker [27] y es llamado de manera asíncrona, desde una herramienta interna de Plusyc denominada Beethoven. El alojamiento remoto del producto se apoyó en los servicios de infraestructura de OCI (por su sigla en inglés, Oracle Cloud Infrastructure)

2.5. Despliegue e integración
[25]. Se provisionó una instancia de cómputo virtual en la nube para desplegar la aplicación y ejecutar los modelos predictivos. El uso de esta tecnología garantiza la disponibilidad continua del sistema. La integración del sistema predictivo requirió del despliegue de una interfaz de programación de aplicaciones. Para su construcción se utilizó FastAPI [25] como marco de trabajo. Esta herramienta facilita el desarrollo de servicios web de alto rendimiento basados en Python [26]. El servicio fue contenedorizado con Docker [27] y es llamado de manera asíncrona, desde una herramienta interna de Plusyc denominada Beethoven. El alojamiento remoto del producto se apoyó en los servicios de infraestructura de OCI (por su sigla en inglés, *Oracle Cloud Infrastructure*) [28]. Se provisionó una instancia de cómputo virtual en la nube para desplegar la aplicación y ejecutar los modelos predictivos. El uso de esta tecnología garantiza la disponibilidad continua del sistema.

Capítulo 3

Diseño e implementación

Este capítulo detalla el diseño y la implementación del sistema predictivo. Se analizan los conjuntos de datos utilizados, la extracción de atributos del audio y el tratamiento de datos según las distribuciones de las variables objetivo. Asimismo, se describen las técnicas de reducción y selección de variables, el proceso de optimización de los modelos y los resultados de la iteración productiva. Finalmente, se presenta la arquitectura desarrollada para el despliegue y la integración del sistema en un entorno operativo.

3.1. Conjuntos de datos y extracción de atributos

Esta sección describe las fuentes de datos analizadas y la metodología para extraer los atributos del audio.

La definición de la estrategia para la extracción de atributos se fundamenta en una revisión de conjuntos de datos públicos estandarizados en el ámbito MIR.

3.1.1. Análisis comparativo de conjuntos de datos públicos

El objetivo de este análisis previo fue entender cómo repositorios de referencia, como FMA y Million Song Dataset [29], representan y estructuran los datos musicales, como método para establecer los criterios de diseño del conjunto de datos propio.

La definición de la estrategia para la extracción de atributos se fundamenta en una revisión de conjuntos de datos públicos estandarizados en el ámbito MIR. El objetivo de este análisis previo fue entender cómo repositorios de referencia, como FMA y Million Song Dataset [29], representan y estructuran los datos musicales, como método para establecer los criterios de diseño del conjunto de datos propio.

En un comienzo se contempló el uso de algunos de estos conjuntos de datos, pero se encontró que presentan varias limitaciones frente a la solución propuesta. Algunos restringen el análisis a segmentos cortos de audio, mientras que otros requieren tamaños de almacenamiento excesivamente pesados o estructuras de datos difíciles de replicar durante el proceso de inferencia y dependientes de las versiones de las bibliotecas de extracción.

Por otro lado, el análisis resultó positivo, estas restricciones motivaron que el proceso de extracción se ejecute directamente sobre el catálogo privado para eliminar las limitaciones y su análisis permitió conocer las variables numéricas en la música, cómo se segmentan las canciones, cómo se reduce la dimensionalidad y hacer los primeros experimentos predictivos de las variables objetivo requeridas.

Por otro lado, el análisis resultó positivo, estas restricciones motivaron que el proceso de extracción se ejecute directamente sobre el catálogo privado para eliminar las limitaciones y su análisis permitió conocer las variables numéricas en la música, cómo se segmentan las canciones, cómo se reduce la dimensionalidad y hacer los primeros experimentos predictivos de las variables objetivo requeridas.

Este conjunto de datos se considera el equivalente al MNIST [30] para el análisis de audio [31], dada su adopción histórica como estándar de evaluación en tareas de clasificación de géneros musicales. Con el análisis de este conjunto de datos se pudo identificar los siguientes puntos:

Este conjunto de datos se considera el equivalente al MNIST [30] para el análisis de audio [31], dada su adopción histórica como estándar de evaluación en tareas

de clasificación de géneros musicales. Con el análisis de este conjunto de datos se pudo identificar los siguientes puntos: [17], descriptores espectrales [32] y coeficientes MFCC.

- Variables predictivas principales: tempo, representaciones chroma [18], amplitud media cuadrática (RMS) [17], descriptores espectrales [32] y coeficientes MFCC.
- Estructura: 1 000 archivos de audio en crudo limitados a 30 segundos, acompañados de metadatos tabulares e imágenes de espectrogramas de Mel en su versión distribuida mediante Kaggle [33]. Todas estas representaciones fueron extraídas con la biblioteca Librosa. Las tablas contienen los atributos calculados sobre la pista completa y también sobre divisiones de 3 segundos para incrementar el volumen de datos de entrenamiento. En ambos escenarios se aplica una reducción de dimensionalidad temporal: las variables se calculan inicialmente sobre micro-ventanas de milisegundos y luego se colapsan a valores estáticos mediante estadísticos de agregación, como la media y la varianza.
- Variables objetivo principales en Plusyc: se podría relacionar con suggested genres.
- Desventajas: provee segmentos cortos de audio, lo que limita la representatividad de la información musical a nivel de canción; y solo abarca 10 géneros musicales, mientras que Plusyc trabaja con más de 500.

FMA estándar

Se analiza la versión estándar de FMA, diseñada para superar las limitaciones de tamaño y acceso a las colecciones completa y mediana. Se encuentra:

Se analiza la versión estándar de FMA, diseñada para superar las limitaciones de tamaño y acceso a las colecciones completa y mediana. Se encuentran:

- Variables predictivas principales: representaciones chroma, Transformada de Q Constante (CQT) [16], tonnetz, MFCC, RMS, descriptores espectrales y tasa de cruces por cero (ZCR).
- Estructura: ofrece recortes de audio de 30 segundos para sus subconjuntos y posteriormente colapsados mediante estadísticos de agregación, como la media, desviación estándar, asimetría y curtosis, para obtener valores estáticos por pista.
- Metadatos contextuales: información tabular adicional que incluye álbum, artista, popularidad, fechas de lanzamiento y ubicación geográfica.
- Variables objetivo principales: géneros musicales estructurados en una taxonomía jerárquica de clases y para un subconjunto de aproximadamente 13 000 pistas, proporciona atributos como *danceability*, *energy*, *valence*, *speechiness* y *liveness*.
- Desventajas: el análisis está restringido a segmentos de 30 segundos en la versión estándar. Las versiones media y completa exceden las capacidades de procesamiento de la infraestructura actual de la empresa. Además, la incertidumbre sobre las versiones exactas de Librosa utilizadas originalmente dificulta la replicabilidad técnica sobre canciones nuevas. Finalmente, la información etiquetada de 13 000 pistas es mucho menor que la de más de 60 000 de Plusyc.

Million Song Dataset (MSD) La replicabilidad técnica sobre canciones nuevas. Finalmente, la información etiquetada de 13 000 pistas es mucho menor que la de más de 60 000 de Plusyc.

Desarrollado por la Universidad de Columbia y The Echo Nest, este conjunto escaló la investigación de información musical a nivel industrial. Se describen sus componentes principales:

Million Song Dataset (MSD)

■ Variables predictivas principales: representaciones de tono (chroma y pitch) [18, 16] y descriptores de timbre [32], ambos calculados a nivel de segmento. Desarrollado por la Universidad de Columbia y The Echo Nest, este conjunto escaló la investigación de información musical a nivel industrial. Se describen sus componentes principales:

■ Estructura: colección de metadatos y descriptores precomputados empaquetados en archivos HDF5.

■ Variables predictivas principales: representaciones de tono (chroma y pitch)

■ Método de extracción: a diferencia [92] de la segmentación por ventanas fijas, el MSD utiliza una división adaptativa basada en eventos. Esto fragmenta la canción en segmentos de duración variable. Para cada uno de estos segmentos, se calculan coeficientes de tono y timbre, y se genera una matriz de atributos alineada con la estructura temporal de la pista.

■ Variables objetivo principales: proporciona atributos musicales fundamentales como tempo, loudness, key y mode.

■ Desventajas: los descriptores se generaron con algoritmos propietarios de The Echo Nest, lo que complica procesar la segmentación por eventos en canciones nuevas de forma idéntica para realizar el proceso de inferencia.

■ Variables objetivo principales: proporciona atributos musicales fundamentales como tempo, loudness, key y mode.

AudioSet Desventajas: los descriptores se generaron con algoritmos propietarios de The Echo Nest, lo que complica procesar la segmentación por eventos en Este conjunto masivo establece un estándar para la clasificación de eventos sonoros generales:

AudioSet Estructura: conjunto de identificadores de videos de YouTube y marcas de tiempo que referencian segmentos de 10 segundos, anotados bajo una ontología de 527 clases. Este conjunto masivo establece un estándar para la clasificación de eventos sonoros generales:

■ Variables predictivas principales: el corpus original no extrae descriptores tradicionales. Su distribución oficial provee embeddings precomputados mediante el modelo VGGish [34], que sirven como entrada rápida para entrenar clasificadores de terceros sin necesidad de descargar el audio original.

■ Variables predictivas principales: el corpus original no extrae descriptores tradicionales. Su distribución oficial provee embeddings precomputados

■ Variables objetivo principales: etiquetas multiclase que identifican la presencia de instrumentos musicales, como tipos de voces (habla, canto) y ambientes acústicos.

■ Desventajas: el enfoque en sonidos generales limita su precisión para los atributos musicales.

■ Variables objetivo principales: etiquetas multiclase que identifican la presencia de instrumentos musicales, como tipos de voces (habla, canto) y ambientes acústicos.

Cabe mencionar que, a pesar de que en algunas de las fuentes de datos públicas se incluye información del contexto como la región del artista, el año de lanzamiento, el álbum o las letras, en esta solución se trabaja exclusivamente con información musical que se extrae del audio.

■ Desventajas: el enfoque en sonidos generales limita su precisión para los atributos musicales.

Cabe mencionar que, a pesar de que en algunas de las fuentes de datos públicas se incluye información del contexto como la región del artista, el año de lanzamiento, el álbum o las letras, en esta solución se trabaja exclusivamente con información musical que se extrae del audio.

En el ámbito del procesamiento de señales, la información de un archivo de audio se representa como una serie de tiempo multivariada: una secuencia de puntos

3.1.2. Reducción de dimensionalidad mediante estadísticas indexados y ordenados cronológicamente.

En el ámbito del procesamiento de señales, la información de un archivo de audio se representa como una serie de tiempo multivariada: una secuencia de puntos de datos [16], en este caso, descriptores numéricos y semánticos, indexados y ordenados cronológicamente. Esto permite reducir estas secuencias a un vector por cada canción, para optimizar el uso de la memoria y los tiempos de procesamiento.

La estrategia adoptada se inspiró en los conjuntos de datos de referencia, donde la señal se divide en ventanas temporales y la información de cada ventana se colapsa en valores estáticos. Esto permite reducir estas secuencias a un vector por cada canción, para optimizar el uso de la memoria y los tiempos de procesamiento.

El diagrama de flujo de este algoritmo se muestra en la figura 3.1, donde se observa cómo la extracción ocurre en ramas paralelas para la transformación de la señal a un vector de estadísticos.

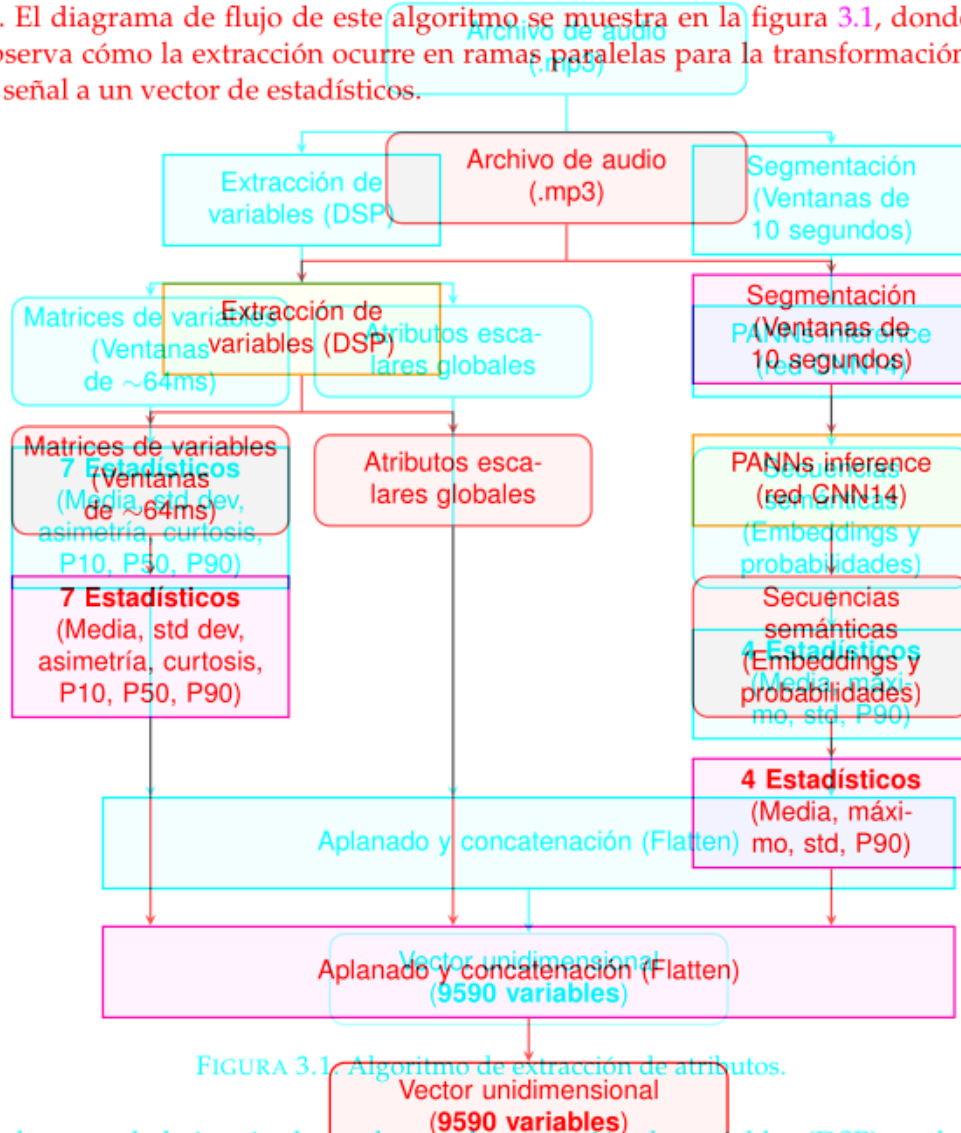


FIGURA 3.1. Algoritmo de extracción de atributos.

En la rama de la izquierda se observa la extracción de variables (DSP) y a la derecha la extracción de embeddings y probabilidades semánticas con PANNs inference:

En la rama de la izquierda se observa la extracción de variables (DSP) y a la derecha la extracción de embeddings y probabilidades semánticas con PANNs inference:

- Para la extracción de variables (DSP) se utilizan las bibliotecas Librosa y Essentia sobre la señal continua, el algoritmo extrae las series temporales de

- las características principales, por ventanas de 64 ms de longitud y solapamiento de 16 ms, que proporciona una alta resolución temporal original de la señal [16, 17]. También se extrae información global de la canción, descriptores escalares para la pista entera que complementan la información segmentada.
- Para la extracción de embeddings y probabilidades se estableció una ventana de análisis de 10 segundos continuos. Esta decisión se fundamenta en la arquitectura de la red neuronal CNN14, entrenada sobre el conjunto de datos AudioSet, cuyos segmentos originales son de 10 segundos. El algoritmo extrae un vector de embeddings y las probabilidades de activación para 173 clases de eventos musicales provistas por PANNs inference por cada ventana.
 - Agrupación estadística: dado que una canción estándar produce una cantidad variable de segmentos, se aplicó una técnica de colapso estadístico a lo largo del eje temporal, según el tipo de variable:
 - Para las variables (DSP), se calcularon siete estadísticos por pista: media, desviación estándar, asimetría, curtosis, y los percentiles 10 (cerca del mínimo), 50 (mediana) y 90 (cerca del máximo para evitar ruido).
 - Para los embeddings y las probabilidades semánticas de las 173 clases seleccionadas de PANNs, se calcularon cuatro medidas: media, valor máximo, desviación estándar y el percentil 90.

Este enfoque matemático permite comprimir el volumen masivo de información de una serie de tiempo en un vector unidimensional por canción. Una representación empírica de este proceso se ilustra en la figura 3.2, donde se observa visualmente cómo las matrices dinámicas de la pista se colapsan en vectores tras el cálculo estadístico. La visualización muestra solo algunos ejemplos, la media en algunos DSP y el percentil 90 en algunas clases del AudioSet.

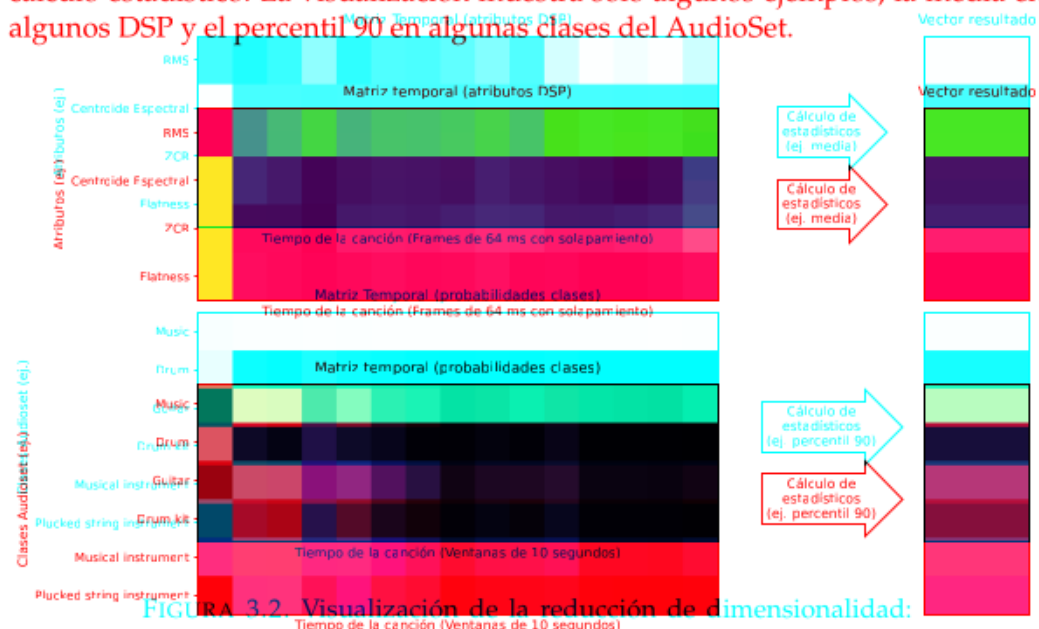


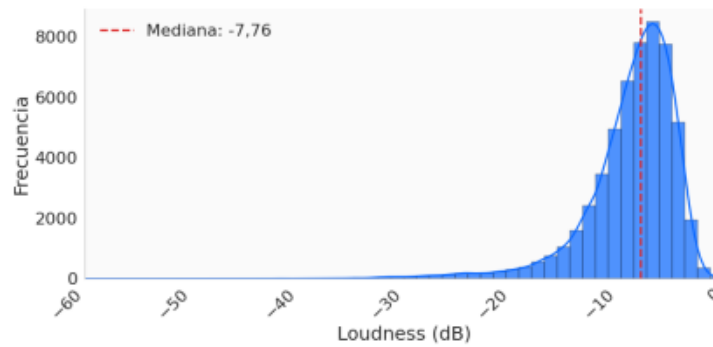
FIGURA 3.2. Visualización de la reducción de dimensionalidad: las secuencias temporales (izquierda) se transforman en vectores

(derecha) mediante la aplicación de medidas de agregación.

FIGURA 3.2. Visualización de la reducción de dimensionalidad: las secuencias temporales (izquierda) se transforman en vectores

(derecha) mediante la aplicación de medidas de agregación.

Algunos ejemplos de la representación permitía capturar la información esencial de la canción de manera eficiente

FIGURA 3.3. Distribución de *loudness* en el catálogo de Plusyc.

Se observan dos desafíos principales inherentes a la distribución:

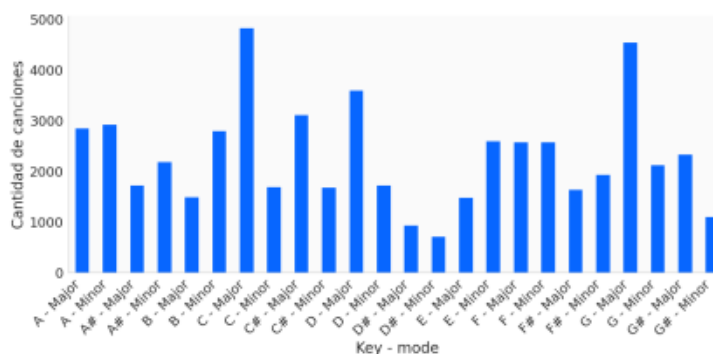
- Existencia de asimetría negativa (sesgo a la izquierda).
- Presencia de valores extremos negativos (hasta -60 dB).

Esto potencialmente podría generar un error matemático que desvíe la atención del algoritmo durante el entrenamiento [24]. Se aplicaron las siguientes transformaciones como parte del preprocesamiento:

- Se aplicó una transformación logarítmica desplazada (Shifted Log Transformation). Primero, se realizó una traslación de la escala (por 60,1) para garantizar que todos los registros fueran estrictamente mayores a cero.
- Posteriormente, se aplicó la función matemática $\log(1 + x)$. Este preprocesamiento comprime la cola izquierda de la distribución, mitiga el sesgo original y estabiliza la varianza para mejorar el rendimiento del modelo de regresión [24].

Key y mode

Las variables *key* y *mode* representan la tonalidad predominante de la pista mapeada a clases de tono enteras (ej. 0 para Do, 1 para Do#) y su modalidad armónica (1 para Mayor, 0 para Menor) [11]. La figura 3.4 muestra la distribución conjunta de las frecuencias de los valores discretos de las 24 tonalidades de la música occidental. Se observa preferencia por tonalidades como Do mayor, Re mayor y Sol mayor; y menor frecuencia de canciones en Re sostenido mayor y menor.

FIGURA 3.4. Distribución conjunta de *key* y *mode* en el catálogo de Plusyc.

Aunque todas comparten un rango de $[0, 1]$, presentan comportamientos distintos: *danceability* muestra una distribución aproximadamente normal; *energy* presenta sesgo negativo; *valence* tiende a la uniformidad; mientras que *acousticness* y de forma extrema, *instrumentalness*, exhiben un sesgo positivo con una concentración masiva de valores cercanos a cero.

El modelado conjunto de múltiples características se aborda convencionalmente mediante redes neuronales, como el Perceptrón Multicapa (MLP), dada su capacidad para aprender de forma simultánea las relaciones complejas y dependencias no lineales entre ellas [24]. Bajo esta premisa, el preprocesamiento requiere prestar atención específica en:

- La heterogeneidad en las formas de las distribuciones y sus varianzas. En arquitecturas sensibles a la escala, como los MLP, las variables con mayor varianza empírica pueden dominar el cálculo de los gradientes, ralentizar o sesgar la convergencia de los pesos [24].
- Las MLP son sensibles a los valores nulos y están presentes en un 6 % de `spectral_entropy_global`.

Se implementaron las siguientes medidas:

- A diferencia del *loudness*, las variables objetivo ya se encuentran acotadas matemáticamente en el intervalo $[0, 1]$. Por tanto, no requirieron transformaciones, ya que este rango es compatible con las funciones de activación continuas, como la sigmoide, en la capa de salida de la red neuronal.
- Se garantizó la integridad de la matriz de características al imputar los nulos residuales con el valor cero y así reflejar la falta de información acústica.
- Finalmente, se aplica estandarización estadística (StandardScaler) a todo el conjunto de entrada para garantizar una media de cero y varianza unitaria ($\mu = 0, \sigma^2 = 1$). Este paso es necesario para homogeneizar el espacio de variables predictivas para el entrenamiento de la red neuronal [24].

Speechiness

El *speechiness* detecta la presencia de palabras habladas en la pista y se define en $[0, 1]$, donde valores altos indican contenido exclusivamente hablado y valores bajos indican música (ej. 0,85 para conversación, 0,05 para música y 0,5 es rap) [11]. La figura 3.7 muestra la distribución de frecuencia del *speechiness*.

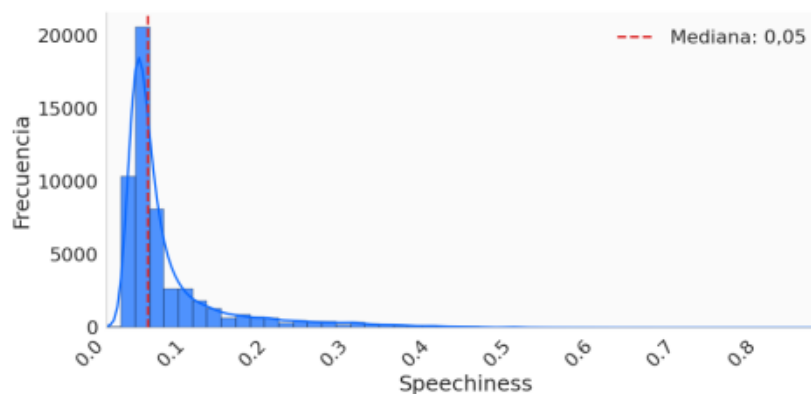


FIGURA 3.7. Distribución de *speechiness* en el catálogo de Plusyc.

- La predicción se fundamenta exclusivamente en variables numéricas extraídas del audio, ya que se carece de información de lenguaje, como las letras de las canciones.
- Ambigüedad; incluso para un humano, algunos de los estados de ánimo pueden ser confusos.

Se aplicaron las siguientes transformaciones como parte del preprocesamiento:

- Se definió con el negocio prescindir de las clases minoritarias *urgent*, *aggressive*, *upbeat* y *fiery* que no son usadas para ambientación.
- Submuestreo de clases mayoritarias para equilibrar la distribución con techo de 2 000 muestras por clase.
- Aplicación de label encoding para traducir las categorías de texto a un formato numérico procesable.
- Para mitigar el desbalance, se aplicó la técnica de ponderación de clases (`class_weights='balanced'`) directamente durante la fase de modelado.

Top-5 secondary mood

El *secondary mood* identifica los estados de ánimo complementarios (ej. *euphoric energy*, *energetic anxious*). La figura 3.10 muestra las frecuencias de los *secondary mood* de Plusyc. Las clases más populares del catálogo son *cheerful / playful* con 4 322 muestras, *happy excitement* con 3 738 y *sweet / sincere* con 3 154.

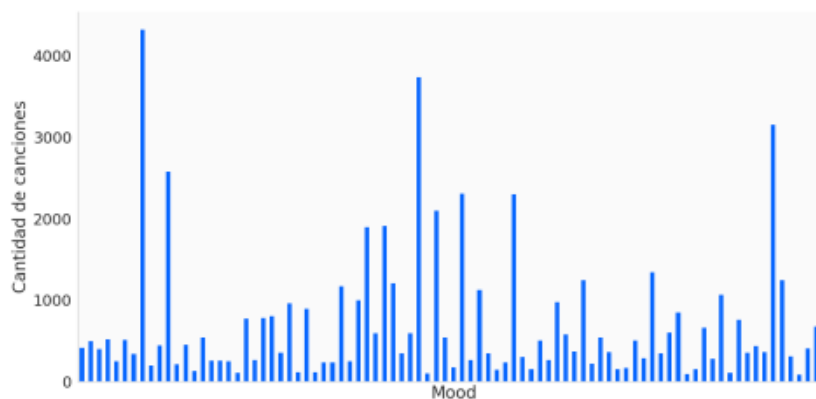


FIGURA 3.10. Distribución de frecuencias de *secondary mood* en el catálogo de Plusyc.

En este caso las dificultades son similares al caso de *primary mood*, en el desbalance de clases, pero con mayor cardinalidad, con 100 clases en total. Además de la delgada frontera en muchos de los estados de ánimo que incluso para el humano resultan confusos.

Medidas aplicadas en el preprocesamiento:

- Se define con el negocio prescindir de las clases minoritarias *aggressive power*, *alienated / brooding*, *bittersweet pop*, *chaotic / intense*, *creepy / ominous*, *energetic dreamy*, *evocative / intriguing*, *focused sparkling*, *hard dark excitement*, *heavy triumphant*, *hypnotic rhythm*, *mysterious / dreamy*, *wild / rowdy* que no son usadas para ambientación en Plusyc.

3.2.2. Selección de variables de entrada

La aplicación del algoritmo de extracción generó un espacio predictivo masivo compuesto por 9590 variables. Modelar directamente sobre esta dimensionalidad introduce riesgo de sobreajuste y alto costo computacional [38, 39]. Por ello, la estrategia de diseño trasladó el esfuerzo hacia el proceso de selección de variables predictivas.

Para todas las variables objetivo, el proceso de selección se ejecutó estrictamente sobre los datos de entrenamiento (X_{train}) para evitar sobreajuste, mediante un flujo secuencial de tres etapas:

1. Filtro por varianza: se implementó `VarianceThreshold` [40] para evaluar cada característica de forma aislada y descartar aquellas constantes o sin varianza (umbral 0,00). Se aplicó un umbral de 0,01 en objetivos de alta cardinalidad para purgar características dispersas que carecen de valor predictivo [41].
2. Selección basada en árboles, con las siguientes características:
 - El cálculo de la importancia se realizó predominantemente mediante un entrenamiento simple, para priorizar la eficiencia computacional. Excepcionalmente, en las variables perceptibles continuas, se implementó validación cruzada (K-Fold con $K = 5$) para mitigar el sobreajuste al ruido en este tipo de etiquetas subjetivas [42].
 - La métrica utilizada fue la ganancia (gain), para medir el aporte de las variables en las particiones de los árboles [43, 22].
 - Se conservaron únicamente las características con mayor importancia, responsables de explicar el 95 % de la ganancia acumulada del modelo.
3. Filtro por dependencia lineal: se calculó la matriz de correlación de Pearson para analizar las características en conjunto y eliminar aquellas que presentaban redundancia lineal [44, 41].

En la tabla 3.4 se resume la configuración del proceso y el total de variables predictivas resultantes para cada variable objetivo.

Tabla 3.4: Resumen de parámetros usados para selección variables.

Variable	Varianza	Validación	Colinealidad	Total
Variable	Varianza	Validación	Colinealidad	Total
Key y mode	0,00	Simple	=	173
Loudness	0,00	Simple	=	856
Perceptibles (5)	0,00	K-Fold ($K = 5$)	$> 0,95$	2140
Speechiness (5)	0,00	K-Fold ($K = 5$)	$\geq 0,95$	2925
Speechiness	0,00	Simple	$\geq 0,95$	76
Primary mood	0,01	K-Fold ($K = 5$)	$\geq 0,95$	1423
Secondary mood	0,01	Simple	$\geq 0,95$	1397
Suggested genres	0,01	Simple	$\geq 0,95$	1309
Suggested genres	0,01	Simple	$> 0,95$	1309

La columna "Varianza", indica el umbral del filtro por varianza, "Validación", indica el tipo de validación utilizada (Simple o K-Fold), y "Colinealidad", indica el umbral de correlación para el filtro por dependencia lineal.

3.3. Modelado y optimización

Para la predicción de las cinco variables perceptibles (*acousticness*, *danceability*, *energy*, *instrumentalness*, *valence*), se implementó una red neuronal de tipo perceptrón multicapa. La arquitectura diseñada consta de tres capas ocultas con una reducción progresiva de neuronas (512, 256 y 128 neuronas, respectivamente). La capa de salida cuenta con cinco neuronas y emplea una función de activación sigmoide, para garantizar que las predicciones se acoten dentro del rango físico [0, 1]. La figura 3.12 ilustra la arquitectura general del perceptrón multicapa implementado.

3.3. Modelado y optimización

Para la predicción de las cinco variables perceptibles (*acousticness*, *danceability*, *energy*, *instrumentalness*, *valence*), se implementó una red neuronal de tipo perceptrón multicapa. La arquitectura diseñada consta de tres capas ocultas con una reducción progresiva de neuronas (512, 256 y 128 neuronas, respectivamente). La capa de salida cuenta con cinco neuronas y emplea una función de activación sigmoide, para garantizar que las predicciones se acoten dentro del rango físico [0, 1]. La figura 3.12 ilustra la arquitectura general del perceptrón multicapa implementado.

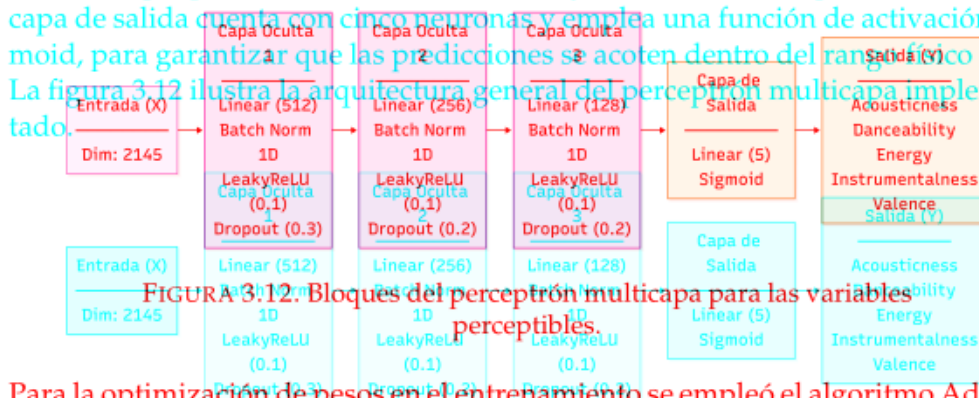


FIGURA 3.12. Bloques del perceptrón multicapa para las variables perceptibles.

Para la optimización de pesos en el entrenamiento se empleó el algoritmo AdamW [45]. Este método incorpora un desacoplamiento de la penalización de peso (weight decay) para favorecer la generalización. Como función de pérdida se seleccionó Huber Loss debido a su robustez frente a valores atípicos en las etiquetas subjetivas [46, 38]. Su ventaja radica en combinar el comportamiento del error absoluto medio (MAE) para penalizar los errores grandes y el del error cuadrático medio (MSE) para los errores pequeños y así asegurar una convergencia suave al acercarse a la solución óptima. Por último, el entrenamiento se ejecutó en lotes de 512 muestras.

Para la optimización de pesos en el entrenamiento se empleó el algoritmo AdamW [45]. Este método incorpora un desacoplamiento de la penalización de peso (weight decay) para favorecer la generalización. Como función de pérdida se seleccionó Huber Loss debido a su robustez frente a valores atípicos en las etiquetas subjetivas [46, 38]. Su ventaja radica en combinar el comportamiento del error absoluto medio (MAE) para penalizar los errores grandes y el del error cuadrático medio (MSE) para los errores pequeños y así asegurar una convergencia suave al acercarse a la solución óptima. Por último, el entrenamiento se ejecutó en lotes de 512 muestras.

Dado el amplio espacio de variables (2.140 variables de entrada) y la alta capacidad del modelo, se implementó la siguiente estrategia de regularización para mitigar el sobreajuste [24]:

- Capas de normalización por lotes (batch normalization) después de cada transformación lineal (Variable) en n5. Las capas, para evitar que los valores numéricos se disparen entre las capas y facilitar la convergencia [47].
- Inactivación aleatoria de neuronas (dropout) con tasas de 0,30 en la primera capa oculta y 0,20 en las subsiguientes, como filtro más agresivo de ruido en las variables iniciales y proteger las representaciones más abstractas en las capas profundas [48].
- Penalización L2 (regularización de Tikhonov) integrada en el optimizador con un factor de 10^{-5} en las subsiguientes, como filtro más agresivo de ruido

- Parada anticipada (early stopping) basada en el conjunto de validación que interrumpió el entrenamiento tras 15 iteraciones sin mejoras.
 - Penalización L2 (regularización de Tikhonov) integrada en el optimizador
- Modelado mediante Gradient Boosting (LightGBM)**

Para la predicción del resto de las variables, se implementaron modelos basados en árboles de decisión potenciados por gradiente, LightGBM [22]. Esta arquitectura se seleccionó por su eficiencia computacional al procesar espacios dimensionales complejos y su capacidad para modelar relaciones no lineales sin requerir transformaciones y tratamiento de valores atípicos en las variables de entrada.

El diseño de los modelos se adaptó a la naturaleza de cada variable objetivo, con base en el preprocesamiento detallado en etapas anteriores:

- *Key / mode, primary, secondary moods y suggested genres*: e configuró la función de pérdida logarítmica multiclase (`multi_logloss`) para la optimización del entrenamiento. Se otorgó mayor peso a las clases minoritarias mediante el parámetro `class_weight='balanced'` y se evaluó el desempeño con la métrica F1-Score macro.

- *Liveness, speechiness*: la separación de los conjuntos de entrenamiento y validación se ejecutó de forma estratificada tras discretizar las variables en rangos probabilísticos.

- *Loudness*: la optimización evaluó el MAE, y el modelado definitivo se configuró mediante regresión estándar (L2) para minimizar el error cuadrático medio (RMSE). Esto demostró una mejora en la predicción en volúmenes extremos como se puede ver en la sección 4.1.2.

La figura 3.13 resume lo anterior.

- *Loudness*: la optimización evaluó el MAE, y el modelado definitivo se configuró mediante regresión estándar (L2) para minimizar el error cuadrático medio (RMSE). Esto demostró una mejora en la predicción en volúmenes extremos como se puede ver en la sección 4.1.2.

La figura 3.13 resume lo anterior.

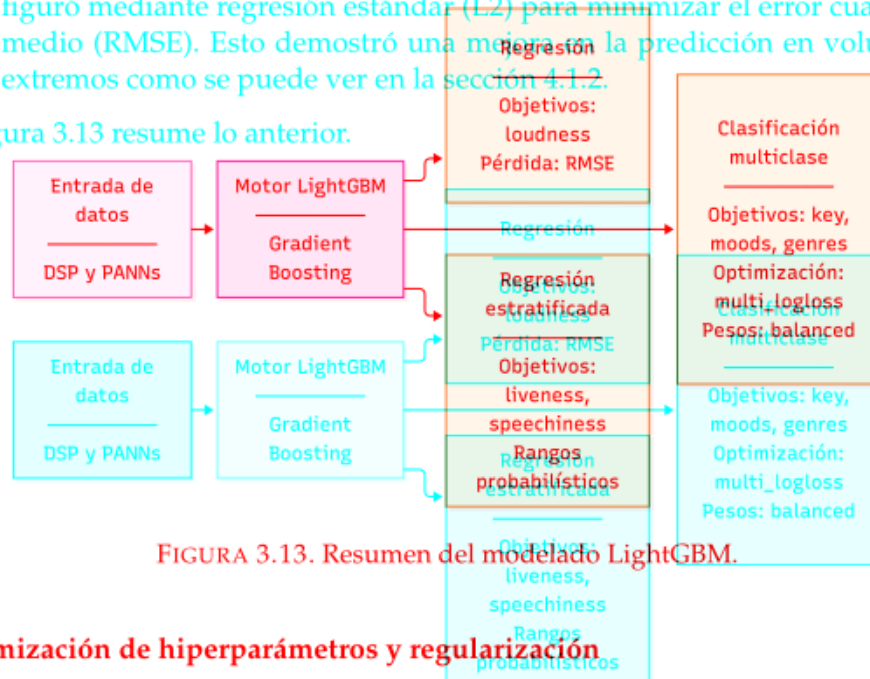


FIGURA 3.13. Resumen del modelado LightGBM.

Optimización de hiperparámetros y regularización

Se implementó una búsqueda automatizada de hiperparámetros para cada objetivo mediante la biblioteca Optuna [23]. A continuación, un resumen de los parámetros y algunas decisiones relacionadas:

- Profundidad máxima de los árboles (`max_depth`) y restricción en el número de nodos (`max_leaf_nodes`)

mero máximo de hojas (`num_leaves`) para evitar el crecimiento asimétrico descontrolado. Además, se exigió un volumen mínimo de muestras por hoja (`min_child_samples` o `min_data_in_leaf`) para evitar que los modelos crearan reglas exclusivas para registros aislados [22].

- Profundidad máxima de los árboles (`max_depth`) y restricción en el número máximo de hojas (`num_leaves`) para evitar el crecimiento asimétrico descontrolado. Además, se exigió un volumen mínimo de muestras por hoja (`min_child_samples` o `min_data_in_leaf`) para evitar que los modelos crearan reglas exclusivas para registros aislados [22].
- Submuestreo de filas y columnas para mejorar la generalización. En la mayoría de los modelos se utilizó el algoritmo (GBDT) combinado con un submuestreo de filas aleatorio (`subsample`). Excepto para *loudness*, donde se implementó el algoritmo de muestreo basado en gradientes (GOSS) [22], para retener ejemplos con mayor error predictivo y descartar una fracción de los que el modelo ya aprendió.
- Penalizaciones (L1 y L2) para frenar el peso excesivo de las ramas [38]. Adicionalmente, en la mayoría de los objetivos, se exigió una mejora mínima (`min_split_gain`) antes de permitir que el árbol se dividiera.
- Early stopping, configurado entre 30 y 50 iteraciones, para interrumpir el entrenamiento cuando el modelo dejara de mejorar.
- Penalizaciones (L1 y L2) para frenar el peso excesivo de las ramas [38]. Adicionalmente, en la mayoría de los objetivos, se exigió una mejora mínima (`min_split_gain`) antes de permitir que el árbol se dividiera.

Una simplificación del flujo de optimización de parámetros se puede ver en la figura (3.14. `split_gain`) antes de permitir que el árbol se dividiera.

- Early stopping, configurado entre 30 y 50 iteraciones, para interrumpir el entrenamiento cuando el modelo dejara de mejorar.

Una simplificación del flujo de optimización de parámetros se puede ver en la figura 3.14.

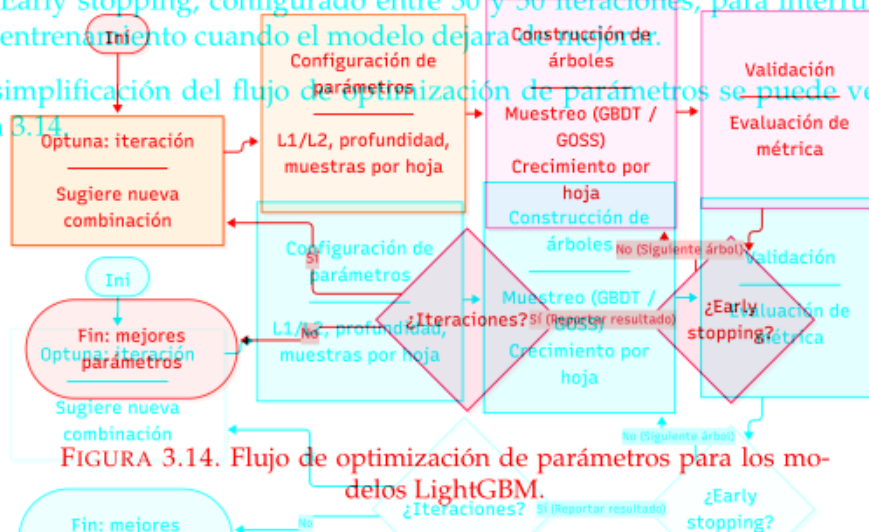


FIGURA 3.14. Flujo de optimización de parámetros para los modelos LightGBM.

El resumen de los hiperparámetros resultantes se detalla en la tabla 3.5. Para el entrenamiento de los modelos finales, se configuró una alta cantidad de árboles (hasta 10 000 en algunos casos) y se delegó el control total al mecanismo de early stopping. Esto, combinado con tasas de aprendizaje muy bajas, mejora el desempeño [38].

3.4. Arquitectura de despliegue e integración

3.4. Arquitectura de despliegue e integración

El sistema de inferencia fue diseñado para operar como un servicio integrable con la infraestructura existente de Plusyc. El despliegue se realizó en una instancia de cómputo sobre OCI.

TABLA 3.5. Resumen de hiperparámetros óptimos por objetivo.

Objetivo	Prof. máx.	Hojas	Tasa de apren.	Muestras/Hoja	L1 (α)	L2 (λ)
Key/Mode	5	140	0,0259	34	3,1690	6,9757
Loudness	10	21	0,0868	25	0,0104	0,0390
Liveness	4	15	0,0194	17	0,0107	0,3887
Speechiness	8	34	0,0102	32	0,4942	0,2947
Primary Mood	10	32	0,0208	79	0,1084	5,0239
Secondary Mood	11	35	0,0125	69	0,1443	2,6547
Genres	6	28	0,0606	21	0,9334	4,8052

3.4.1. Diseño del servicio predictivo e integración

El núcleo de procesamiento se expuso a través de una API RESTful desarrollada con FastAPI. Esta interfaz expone el ciclo de inferencia: carga del audio, extracción de variables predictivas de entrada a los modelos MLP y LightGBM, que retornan los valores de las predicciones de las características musicales. Para garantizar la reproducibilidad y estabilidad del entorno, la aplicación fue contenedorizada utilizando Docker, como se observa en la figura 3.15.

3.4.1.1. Diseño del servicio predictivo

El núcleo de procesamiento se expuso a través de una API RESTful desarrollada con FastAPI. Esta interfaz expone el ciclo de inferencia: carga del audio, extracción de variables predictivas de entrada a los modelos MLP y LightGBM, que retornan los valores de las predicciones de las características musicales. Para garantizar la reproducibilidad y estabilidad del entorno, la aplicación fue contenedorizada utilizando Docker, como se observa en la figura 3.15.

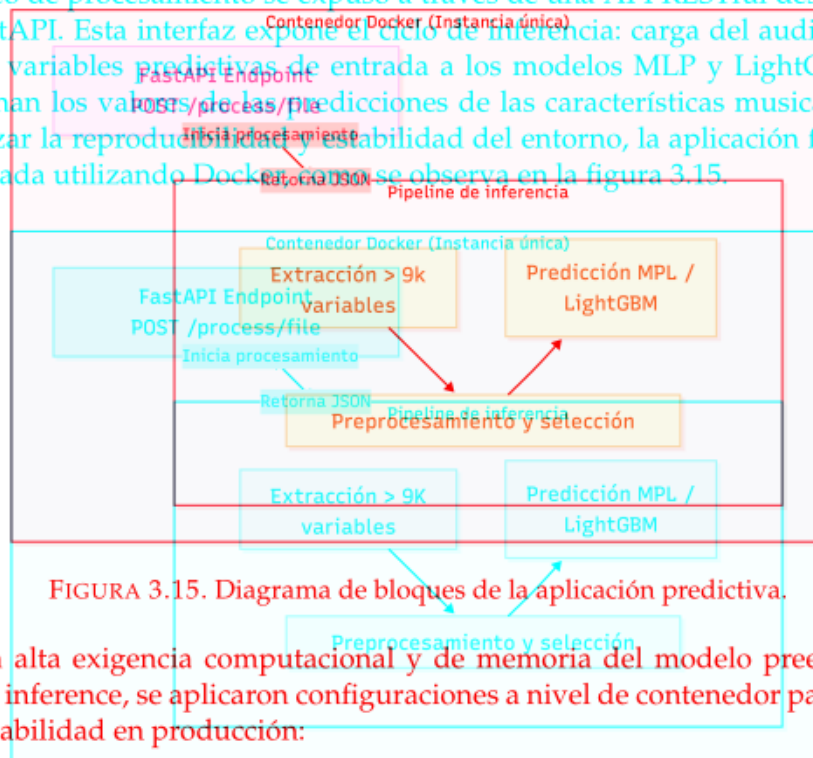


FIGURA 3.15. Diagrama de bloques de la aplicación predictiva.

Dada la alta exigencia computacional y de memoria del modelo preentrenado PANNs inference, se aplicaron configuraciones a nivel de contenedor para asegurar la viabilidad en producción:

- Se limitó a una instancia. Esto previene la duplicación masiva de los modelos y pesos preentrenados en la memoria RAM, para optimizar el consumo de los recursos de la máquina virtual y evitar nuevos costos [49].

Dada la alta exigencia computacional y de memoria del modelo preentrenado PANNs inference, se aplicaron configuraciones a nivel de contenedor para asegurar la viabilidad en producción:

- Restricción de hilos: las operaciones matemáticas requeridas para la extracción de variables se restringieron a un solo hilo (`OMP_NUM_THREADS=1`). Esto evita cuellos de botella por saturación de la CPU y bloqueos cruzados (deadlocks) durante la extracción paralela de variables [50].
- Se limitó a una instancia. Esto previene la duplicación masiva de los modelos y pesos preentrenados en la memoria RAM, para optimizar el consumo de los recursos de la máquina virtual y evitar nuevos costos [49].

- Gestión de memoria y tiempo de respuesta: se asignó un bloque de memoria compartida (`shared_size: '5gb'`) y se establecieron timeouts holgados (120 segundos) para prevenir interrupciones provocadas por el análisis de pistas de audio de larga duración, restringidas a 15 máximo por canción.

3.4.2. Flujo de integración

La interacción del motor predictivo con el ecosistema web se realiza a través de un flujo de procesamiento asíncrono. La comunicación entre componentes es orquestada por una herramienta de Plusyc denominada Beethoven.

3.4.2. Flujo de integración

El ciclo de vida de la integración, ilustrado en la figura 3.16, sigue esta secuencia lógica:

1. Una tarea programada en el sistema ejecuta el CLI Beethoven a intervalos regulares de tiempo.

El ciclo de vida de la integración, ilustrado en la figura 3.16, sigue esta secuencia lógica:

2. El proceso detecta nuevos archivos de audio que aún no han sido caracterizados.
3. Una tarea programada en el sistema ejecuta el CLI Beethoven a intervalos regulares de tiempo.
3. Por cada nueva pista detectada, Beethoven envía el archivo al servicio de inferencia (API) mediante una petición HTTP POST.
2. El proceso detecta nuevos archivos de audio que aún no han sido caracterizados.
4. Inferencia: la API ejecuta el pipeline predictivo, que recibe la señal cruda y retorna un diccionario estructurado (JSON) que contiene todas las etiquetas estimadas (`key, moods, liveness, etc.`).
3. Beethoven envía el archivo al servicio de inferencia (API) mediante una petición HTTP POST.
5. El script captura la respuesta de la API, extrae las características musicales y las persiste en la base de datos de información musical de Plusyc.
4. La API ejecuta el pipeline predictivo, que recibe la señal cruda y retorna un diccionario estructurado (JSON) que contiene todas las etiquetas estimadas (`key, moods, liveness, etc.`).
6. Una vez en la base de datos, los metadatos quedan disponibles para revisión de curadores en un portal web.
5. El script captura la respuesta de la API, extrae las características musicales y las persiste en la base de datos de información musical de Plusyc.

6. Una vez en la base de datos, los metadatos quedan disponibles para revisión de curadores en un portal web.

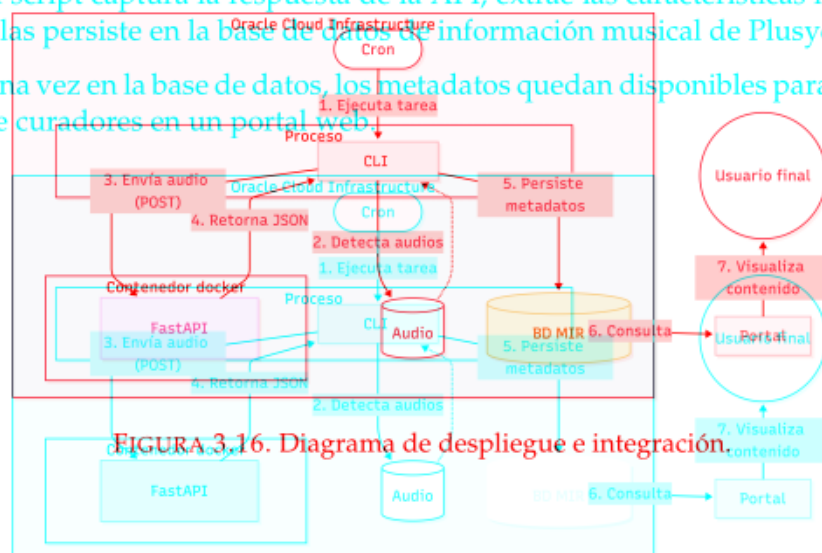


FIGURA 3.16. Diagrama de despliegue e integración.

FIGURA 3.16. Diagrama de despliegue e integración.

20, respectivamente. A partir de este punto, la validación se mantiene constante mientras el error de entrenamiento presenta un descenso marginal y evidencian aprendizaje sin sobreajuste.

La tabla 4.1 muestra cómo la inclusión de embeddings y probabilidades de PANNs inference y la separación de *speechiness* y *liveness* mejoró consistentemente todas las métricas, con R^2 global de 0,8152.

TABLA 4.1. Comparativa de métricas de dos experimentos de la red neuronal MLP.

Variable	R^2 ($\uparrow$)		MAE ($\downarrow$)		MSE ($\downarrow$)	
	Inicial	Definitivo	Inicial	Definitivo	Inicial	Definitivo
<i>Acousticness</i>	0,8178	0,8903	0,0950	0,0681	0,0171	0,0106
<i>Danceability</i>	0,6042	0,7381	0,0780	0,0594	0,0108	0,0063
<i>Energy</i>	0,7584	0,8626	0,0809	0,0581	0,0110	0,0062
<i>Instrumentalness</i>	0,6733	0,8308	0,1231	0,0671	0,0355	0,0188
<i>Valence</i>	0,6523	0,7539	0,1172	0,0964	0,0221	0,0162
<i>Liveness</i>	0,1570	–	–	–	–	–
<i>Speechiness</i>	0,2778	–	–	–	–	–

La figura 4.2 muestra los resultados de los valores reales de las etiquetas y las estimaciones del modelo. En estos gráficos de dispersión, la línea diagonal de referencia representa el escenario de predicción perfecta. La proximidad y densidad de los puntos respecto a esta recta ilustran la relación lineal, la varianza y la capacidad del modelo frente a datos no vistos.

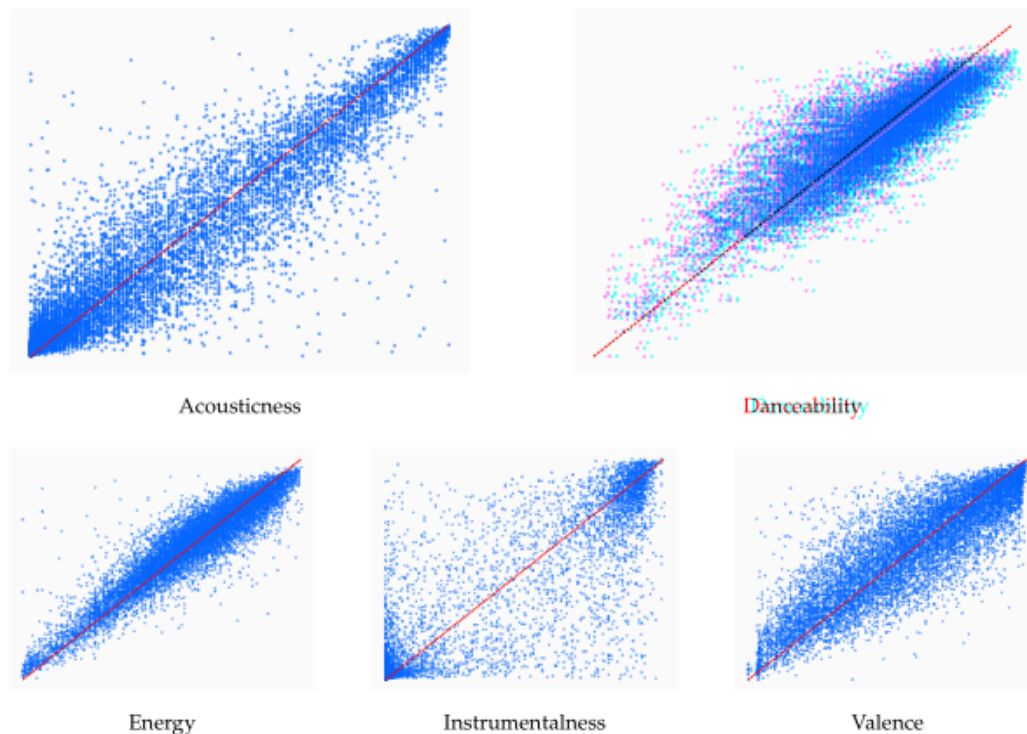


FIGURA 4.2. Relación de valores reales y estimados en el modelo MLP.

Una lista de importancia de variables predictivas muestra predominio de las variables globales y ritmo presentadas en la tabla 3.1. La figura 4.3 presenta las 15 variables más relevantes del modelo MLP. En la figura, el grado de importancia equivale a la caída en el coeficiente de determinación (R^2) que sufre el modelo al eliminar la información de dicha variable. Se destaca el impacto principal del factor de cresta, representaciones chroma y estadísticos de onset. También es interesante la ausencia de embeddings y probabilidades de PANNs en el top-15, pero muchas están presentes entre las variables seleccionadas (2 140 en total), lo que muestra que operan en conjunto. El anexo A muestra la proporción total de cada tipo de variable por modelo.

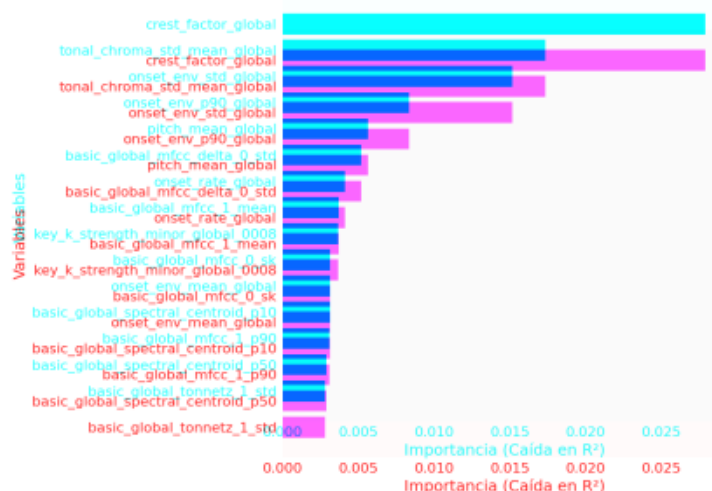


FIGURA 4.3. Top-15 variables predictivas del modelo MLP.

FIGURA 4.3. Top-15 variables predictivas del modelo MLP.

4.1.2. Loudness

4.1.2. Loudness

La inferencia de *loudness* consistió en un modelo de regresión LightGBM. En la configuración inicial se probó con penalización L1 (regresión de Lasso). La figura 4.4 expone las curvas de aprendizaje para este experimento, donde se observa el descenso de los errores de entrenamiento y validación a medida que se suman árboles al modelo. La trayectoria y estabilización conjunta de ambas curvas evidencian un aprendizaje estable.

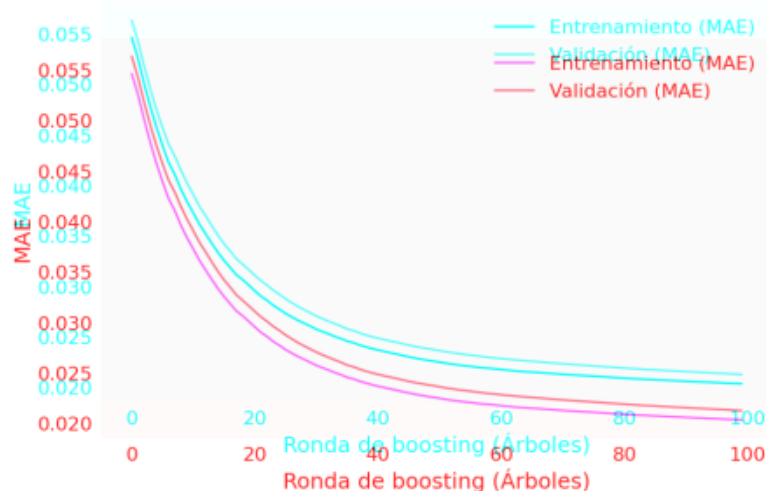


FIGURA 4.4. Curvas de aprendizaje para loudness.

FIGURA 4.4. Curvas de aprendizaje para loudness.

4.1.3. Key y mode

Se implementó un modelo LightGBM que optimiza la pérdida logarítmica multi-clase (`multi_logloss`). Durante el entrenamiento, la función de early stopping detuvo el proceso en la iteración 736 al estabilizarse la pérdida de validación en 1,1926.

La evaluación del modelo alcanzó una exactitud (accuracy) de 0,6106 y un F1-Score ponderado de 0,6115. Destaca un AUC-ROC promedio de 0,9589, lo que indica buena capacidad de discriminación entre las 24 clases. Adicionalmente, el accuracy del top-3 alcanzó 0,8671, donde la tonalidad correcta se encuentra casi siempre entre las tres primeras predicciones del sistema. La figura 4.7 presenta la matriz de confusión absoluta.

Valores actuales	C Minor	168	8	1	11	8	10	0	0	1	3	0	21	1	0	35	4	0	0	34	3	0	28	1	1
	C Major	13	626	1	14	7	5	6	3	33	12	0	0	29	1	12	53	0	3	24	35	0	4	80	3
	C# Minor	1	0	192	12	2	2	4	25	0	3	1	0	3	25	1	3	31	2	1	3	18	1	0	5
	C# Major	1	9	9	362	55	1	3	7	0	6	16	0	0	0	55	4	8	30	0	9	10	36	0	1
	A# Minor	5	4	1	57	245	4	3	1	1	3	13	0	0	0	34	4	2	15	3	3	2	31	3	2
	A# Major	11	4	0	1	13	231	0	1	11	2	0	7	0	1	12	12	0	2	27	4	0	1	1	2
	B Minor	0	3	4	2	6	1	337	12	0	54	0	0	44	4	1	0	43	2	2	20	2	1	2	18
	B Major	0	0	11	7	7	0	6	192	1	0	6	2	4	7	1	0	8	13	0	2	26	1	1	2
	D Minor	2	22	0	3	1	5	1	1	218	10	0	1	1	1	0	29	1	0	20	5	0	1	21	0
	D Major	2	2	5	6	1	2	63	2	16	478	0	1	15	1	0	0	18	2	15	43	1	3	15	27
	D# Minor	0	0	0	3	17	2	0	12	1	2	65	2	0	0	2	1	0	19	0	0	13	1	0	0
	D# Major	11	0	0	1	5	6	0	0	1	2	0	121	1	2	5	1	0	2	6	2	1	16	1	0
	E Minor	0	11	2	1	0	2	50	1	1	23	0	0	301	5	0	0	10	4	0	52	0	1	40	13
	E Major	0	1	34	1	2	1	4	10	1	2	0	0	11	193	0	2	7	1	1	1	7	0	2	14
	F Minor	32	6	0	27	38	16	0	0	5	1	1	14	1	1	272	9	2	5	23	3	0	52	4	1
	F Major	2	32	1	6	4	19	3	0	45	5	0	1	5	2	11	324	0	2	20	7	0	1	21	3
	F# Minor	2	3	39	11	1	0	25	4	1	6	1	0	4	17	2	1	220	6	0	4	10	0	2	27
	F# Major	0	3	6	27	27	1	5	11	0	1	14	0	1	0	4	2	13	187	1	3	18	1	0	2
	G Minor	14	14	1	3	3	34	0	0	25	7	0	8	0	1	7	15	1	1	255	12	0	7	13	2
	G Major	0	51	1	7	4	0	36	1	12	54	0	1	75	0	10	5	3	7	16	562	1	10	51	0
	G# Minor	0	0	15	18	4	0	3	29	0	1	12	2	2	2	0	2	4	12	2	3	101	2	1	3
	G# Major	40	2	5	19	23	4	3	3	0	6	0	18	0	1	33	2	5	0	14	6	2	280	0	0
	A Minor	2	47	0	3	3	2	5	0	23	14	1	0	36	4	0	10	1	1	3	27	0	1	389	10
	A Major	0	3	20	4	0	3	24	3	1	22	0	0	11	12	0	5	33	0	0	6	2	3	23	394
Predicciones		C Minor	C Major	C# Minor	C# Major	A# Minor	A# Major	B Minor	B Major	D Minor	D Major	D# Minor	D# Major	E Minor	E Major	F Minor	F Major	F# Minor	F# Major	G Minor	G Major	G# Minor	G# Major	A Minor	A Major

FIGURA 4.7. Matriz de confusión de key y mode.

El análisis de los errores revela una fuerte coherencia armónica cuando el modelo se equivoca en la predicción principal con datos no vistos, las confusiones más frecuentes ocurren con la tonalidad relativa (mayor o menor correspondientes) o con las tonalidades dominante y subdominante de la escala mayor o menor correspondiente. El anexo B desglosa los resultados de las relaciones más importantes. Para contextualizar este desempeño, la tabla 4.2 contrasta los resultados del extractor de tonalidad de la biblioteca Essentia evaluado en el dataset de Plusyc,

Los valores del área bajo la curva (AUC) demuestran capacidad de discriminación en la mayoría de los casos. Destacan clases como *peaceful* (0,94) y *tender* (0,88) con alta separabilidad, frente a *lively* (0,68). El algoritmo logra modelar distribuciones de probabilidad a pesar de la ambigüedad de las etiquetas.

El modelo de *secondary mood*, con 102 clases, evidenció un comportamiento similar. El accuracy directo fue del 18,75 % con un F1-Score macro de 0,1458. No obstante, al evaluar las sugerencias, el accuracy subió progresivamente al 46,57 % en el top-5 y a 79,00 % recién en el top-20. la figura 4.20 ilustra la matriz de confusión normalizada de *secondary mood*, donde también se muestra una diagonal principal que indica aprendizaje. Y algunos indicios de líneas verticales muestran tendencia a preferir las clases predominantes.

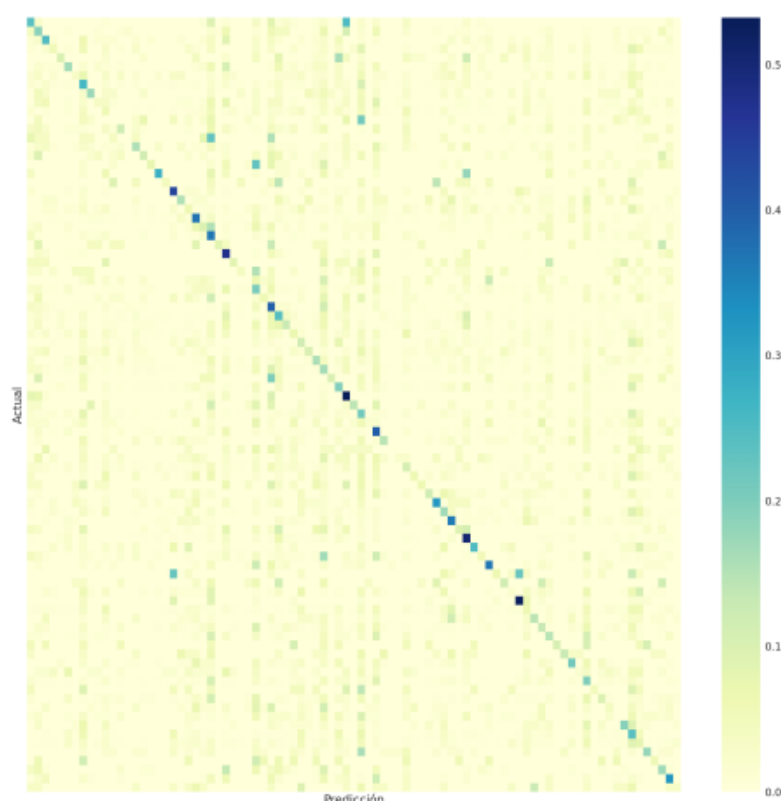


FIGURA 4.20. Matriz de confusión normalizada de secondary mood.

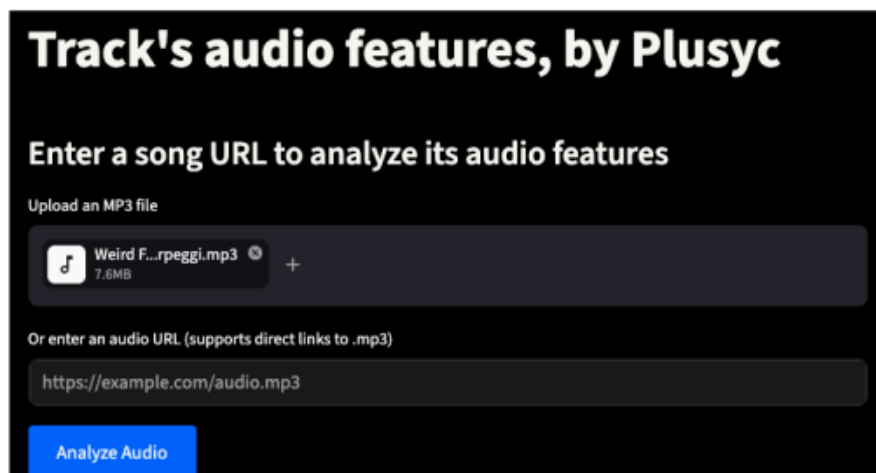
Aunque las métricas estrictas son bajas en comparación con otras variables analizadas, la evolución positiva en las precisiones top-K confirma que ambos modelos lograron extraer señal de las variables de entrada y que las predicciones se agrupan consistentemente dentro de espectros de estados de ánimo similares. Este comportamiento resulta útil en el contexto de Plusyc.

Secondary mood está presente como característica pero actualmente no se usa como un filtro para ambientación en Plusyc. Esto también explica el desbalance, los curadores tienden a favorecer algunos estados de ánimo secundarios más que otros.

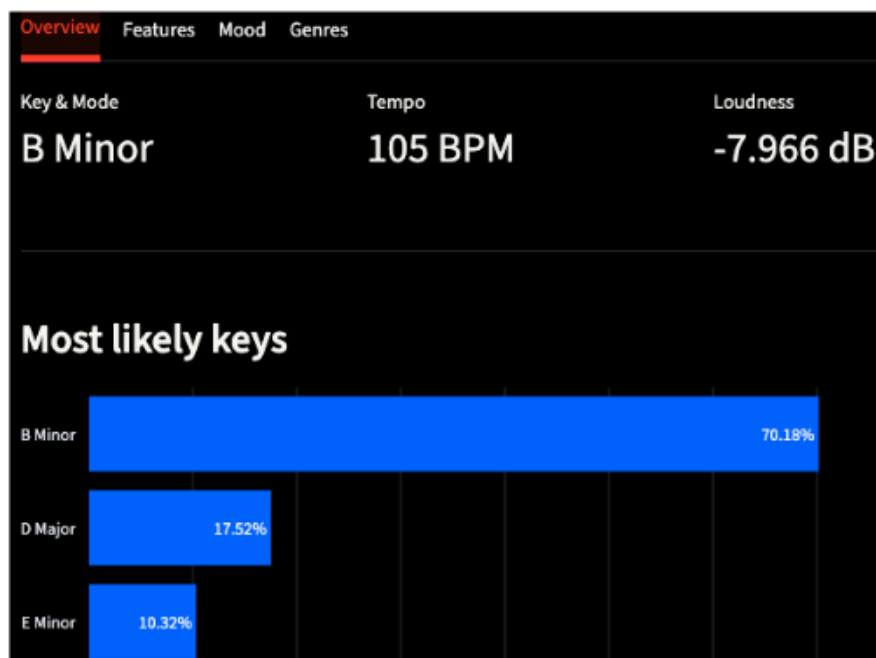
4.2. Ejemplo de inferencia

Para demostrar el funcionamiento integral del sistema, se ejecutó una inferencia cuya entrada es la canción de Radiohead ilustrada en los primeros capítulos y que no está en el catálogo de Plusyc. El archivo de audio se procesó a través de la API. En el anexo [se presenta la respuesta](#).

Las figuras 4.21, 4.22 y 4.23 exponen una representación visual de estos resultados en una interfaz de usuario. En la primera parte, la figura 4.21a ilustra la pantalla de selección de la canción procesada, seguida por el resumen general en la figura 4.21b, donde aparecen las estimaciones de *key*, *mode*, *tempo* y *loudness*, e incluye el top-3 de *key* y *mode* en la parte inferior.



(A) Pantalla de selección de la canción.



(B) Resumen general de resultados, características básicas.

FIGURA 4.21. Visualización de los resultados de inferencia en una interfaz web (parte 1).

con el conjunto de datos AudioSet, creado para la generación automática de subtítulos como [Aplausos] o [Rapping].

- Completar el trabajo tomó más tiempo del planificado en el cronograma original. Hubo una subestimación del esfuerzo, dado que la cantidad de modelos y el proceso de aprendizaje requirieron más iteraciones de las previstas.

5.2. Próximos pasos

Los diseños actuales quedan disponibles y se pueden usar para iteraciones futuras. Algunos aspectos que se podrían mejorar o explorar son:

- El uso de las inferencias de PANNs y la exploración de otros modelos preentrenados, para mejorar la limpieza de algunos datos y corregir audios mal etiquetados. Como ejemplos, el *speechiness*, *liveness* y la predicción de *instrumentalness*: el modelo actual logra diferenciar una canción instrumental de una que no lo es, pero los valores en el medio no son claros. Se sugiere revisar puntualmente, ya que presenta ruido en los valores intermedios por falta de definiciones claras.
- También se sugiere iterar sobre el *tempo* para buscar una solución más robusta que el algoritmo de DSP.
- Se propone empezar a usar el estado de ánimo secundario como filtro en Plusyc para mejorar la precisión de las recomendaciones musicales.
- El sistema se puede ajustar para el despliegue bajo el modelo de software como servicio y así permitir ofrecer el servicio al público interesado.
- El equipo de curadores sigue clasificando música. Se contempla automatizar un reentrenamiento periódico con Airflow para que el sistema aprenda nuevos géneros e integrar MFlow para gestionar el ciclo de vida operativo. Esto permitirá establecer un registro centralizado de modelos, gestionar su versionado, controlar la promoción y devolución entre entornos.
- Si bien empíricamente las inferencias muestran alta coherencia entre el *primary* y *secondary mood*, actualmente son generadas por modelos independientes. Se sugiere relacionar las dos variables e implementar una arquitectura de clasificación jerárquica.
- Se podría evaluar a futuro el uso de arquitecturas basadas en transformadores, si se cuenta con más muestras y mayor capacidad de cómputo, para mejorar la clasificación de clases densas. También, complementar el sistema con modelos de aprendizaje profundo enfocados exclusivamente en música. Sin embargo, los resultados de este trabajo reafirman que la aplicación de técnicas tradicionales de aprendizaje automático sigue ofreciendo soluciones robustas.

Apéndice A

Apéndice A: Proporción de variables predictivas por modelo

Se detalla la cantidad y proporción de los tipos de variable predictiva.

1 {
2 "success": true,
3 "data": {
4 "key": {
5 "value": {
6 { "key": 11, "mode": 0, "key_mode": "B Minor", "prob": "70.18%"
7 { "key": 2, "mode": 1, "key_mode": "B Major", "prob": "17.32%"
8 { "key": 4, "mode": 0, "key_mode": "B Minor", "prob": "0.00%" }
9 }
10 { "key": 11, "mode": 0, "key_mode": "B Minor", "prob": "70.18%"
11 { "key": 2, "mode": 1, "key_mode": "B Major", "prob": "17.32%"
12 { "key": 4, "mode": 0, "key_mode": "B Minor", "prob": "0.00%" }
13 }
14 { "key": 11, "mode": 0, "key_mode": "B Minor", "prob": "70.18%"
15 { "key": 2, "mode": 1, "key_mode": "B Major", "prob": "17.32%"
16 { "key": 4, "mode": 0, "key_mode": "B Minor", "prob": "0.00%" }
17 }
18 }
19 }
20 }
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }

TABLA A.1. Distribución del tipo de variables predictivas en los modelos.

Variable objetivo	Total	DSP	PANNs	Emb.	Prob.	Total (%)
Key y mode	173	173 (100,0 %)	0	0	0	0 (0,0 %)
Perceptibles (MLP)	2 140	480 (22,4 %)	1 315	345	1 660	(77,6 %)
Speechiness	92	22 (23,9 %)	62	8	70	(76,1 %)
Liveness	76	24 (31,6 %)	36	16	52	(68,4 %)
Primary mood	1 423	382 (26,8 %)	1 010	31	1 041	(73,2 %)
Secondary mood	1 397	385 (27,6 %)	980	32	1 012	(72,4 %)
Suggested genres	1 309	370 (28,3 %)	912	27	939	(71,7 %)

"liveness": { "value": 0.116 },
"Speechiness": { "value": 0.035 },
"MoodPrimary": {
"top_5":
"Yearning 18.20%, Stirring 13.95%, Melancholy 9.87%, Brooding:
8.87%, Rowdy:7.67%"
},
"MoodSecondary": {
"top_5":
"Cool Melancholy 6.30%, Lyrical Sentimental 4.89%, Awakening /
Stately 4.64%, Light Melancholy 4.55%, Gentle Bittersweet 4.17%"
},
"GenreTarget": {
"top_5":
"Mainstream 90s and 00s Rock, Alternative Rock, General Latin
Rock, General Downtempo, General Indie Rock"
}
}
}
}
}
}

CÓDIGO A.1. Respuesta JSON de inferencia.

Apéndice B

Apéndice B: desglose armónico

[1] J. Stephen Downie. «Music Information Retrieval». En: *Annual Review of Information Science and Technology* (2003).

[2] Michael A. Casey et al. «Content-Based Music Information Retrieval: Current Directions and Future Challenges». En: *Proceedings of the IEEE* (2008).

[3] Gordon C Bruner II. «Music, mood, and marketing». En: *Journal of marketing* (1990).

[4] Elanor V. Genthie y Keri D. S. «Setting the atmosphere with the music: A meta-analytic review of the effects of background music in retail settings». En: *Journal of Business Research* (2008).

[5] Holger Roschk, Sandra Maria Correia Loureiro y Jan Breitsohl. «Calibrating 30 years of experimental research: a meta-analysis of the atmospheric effects of music, scent, and color». En: *Journal of Retailing* (2017).

[6] Plusyc Live SAS. *Sitio web oficial*. 2026. URL: <https://www.plusyc.com/>.

[7] Brian McFee et al. «Librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference*. Visitado el 2026-03-20. URL: <http://librosa.org/>.

[8] Dmitry Bogdanov et al. «Essentia: An Audio Analysis Library for Music Information Retrieval». En: *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)* 2013. URL: <http://essentia.upf.edu/>.

[9] Jordi Pons y Xavier Serra. «musicnn: Pre-trained Convolutional Neural Networks for Music Audio Tagging». En: *Late-Breaking/Demo Session of the 20th International Society for Music Information Retrieval Conference (ISMIR)* 2019. URL: <https://github.com/jordipons/musicnn>.

[10] Jort F. Gemmeke et al. «Audio Set: An ontology and human-labeled dataset for audio events». En: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2017. URL: <https://research.google.com/audioset/>.

[11] Spotify. *Spotify Web API Reference: Get Audio Features*. 2026. URL: <https://developer.spotify.com/documentation/web-api/reference/get-audio-features/>.

[12] Gracenote Inc. *Gracenote Music Data: Sonic Attributes and Mood*. 2026. URL: <https://gracenote.com/products/audio-data/>.

[13] Tunebat. *Tunebat: Song Key, BPM and Audio Features Database*. 2026. URL: <https://tunebat.com/>.

[14] George Tzanetakis y Perry Cook. «Musical genre classification of audio signals». En: *IEEE Transactions on Speech and Audio Processing* (2002).

[15] Michaël Defferrard et al. «FMA: A Dataset for Music Analysis». En: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017.

[16] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

[17] Alexander Lerch. *An introduction to audio content analysis: Applications in signal processing and music informatics*. John Wiley & Sons, 2012.

Tonalidad	Exacta (%)	Relativa (%)	Dominante (%)	Subdom. (%)	Paralela (%)	Suma (%)
C Minor	49,7	6,2	10,1	10,4	2,4	78,8
C Major	64,9	8,3	3,6	5,5	1,3	83,6
C# Minor	57,3	7,5	5,4	9,3	3,6	83,1
C# Major	58,2	8,8	5,8	4,8	1,4	79,0
A# Minor	56,2	13,1	7,8	3,0	0,9	81,0
A# Major	67,3	7,9	5,5	2,0	3,8	84,5
B Minor	60,4	9,7	7,7	7,9	2,2	87,9
B Major	64,6	8,8	4,4	2,4	0,0	80,2
D Minor	63,6	8,5	6,1	5,8	2,9	86,9
D Major	66,6	8,8	3,8	6,0	2,2	87,4
D# Minor	46,4	13,6	12,1	9,3	1,4	82,8
D# Major	65,8	6,0	3,3	8,7	0,0	83,8
E Minor	58,2	10,0	0,2	7,7	1,0	77,1
E Major	65,4	11,5	3,4	4,7	3,7	88,7
F Minor	53,0	10,1	6,2	7,4	1,8	78,5
F Major	63,0	8,8	6,2	3,7	2,1	83,8
F# Minor	57,0	7,0	10,1	6,5	1,6	82,2
F# Major	57,2	4,3	8,3	5,4	4,0	77,2
G Minor	60,3	8,0	5,9	3,3	1,7	79,2
G Major	62,0	8,3	6,0	5,6	4,0	85,9
G# Minor	46,3	13,3	5,5	6,9	0,9	72,9
G# Major	60,1	7,1	3,9	4,1	0,0	75,2
A Minor	66,8	8,1	6,2	4,0	1,7	86,8
A Major	69,2	5,8	2,1	3,9	4,0	85,0

- [18] Emilia Gómez. «Tonal description of polyphonic audio for music content processing». En: *INFORMS Journal on Computing* (2006).
- [19] Christopher Harte, Mark Sandler y Martin Gasser. «Detecting harmonic change in musical audio». En: *Proceedings of the 1st ACM workshop on Audio and music computing multimedia*. 2006.
- [20] Qiuqiang Kong et al. «PANNS: Large-scale pretrained audio neural networks for audio pattern recognition». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2020).
- [21] Karen Simonyan y Andrew Zisserman. «Very deep convolutional networks for large-scale image recognition». En: *International Conference on Learning Representations*. 2015.
- [22] Guolin Ke et al. «Lightgbm: A highly efficient gradient boosting decision tree». En: *Advances in neural information processing systems*. Vol. 30. 2017.
- [23] Takuya Akiba et al. «Optuna: A next-generation hyperparameter optimization framework». En: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019.
- [24] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep learning*. MIT press, 2016.
- [25] Sebastián Ramírez. *FastAPI*. 2018. URL: <https://github.com/fastapi/fastapi>.
- [26] Python Software Foundation. *Python Language Reference*. 2026. URL: <https://www.python.org/>.
- [27] Dirk Merkel. «Docker: lightweight linux containers for consistent development and deployment». En: *Linux journal* (2014).
- [28] Oracle Corporation. *Oracle Cloud Infrastructure*. 2026. URL: <https://www.oracle.com/cloud/>.
- [29] Thierry Bertin-Mahieux et al. «The million song dataset». En: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*. 2011.
- [30] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* (1998).
- [31] Bob L. Sturm. «The state of the art ten years after a state of the art: Future research in music information retrieval». En: *Journal of New Music Research* (2014).
- [32] Geoffroy Peeters. *A large set of audio features for sound description (similarity and classification) in the CUIDADO project*. Inf. téc. IRCAM (Institut de Recherche et Coordination Acoustique/Musique), 2004.
- [33] Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. 2020. URL: <https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>.
- [34] Shawn Hershey et al. «CNN architectures for large-scale audio classification». En: *IEEE*. 2017.
- [35] ITU-R. «Algorithms to measure audio programme loudness and true-peak audio level». En: *Recommendation ITU-R BS.1770-4* (2015).
- [36] Alain De Cheveigné e Hideki Kawahara. «YIN, a fundamental frequency estimator for speech and music». En: *The Journal of the Acoustical Society of America* (2002).
- [37] Earl Vickers. «The loudness war: Background, speculation and recommendations». En: *Audio Engineering Society*. 2010.

[38] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.

[39] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[40] Fabian Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* (2011).

[41] John D. Fienberg y Mark J. Johnson. *Symbolic Statistical Modeling*. Springer, 2013.

[42] Ron Kohavi. «A study of cross-validation and bootstrap for accuracy estimation and model selection». En: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*. Morgan Kaufmann Publishers Inc. 1995.

[43] Leo Breiman et al. *Classification and Regression Trees*. CRC press, 1984.

[44] Isabelle Guyon y André Elisseeff. «An introduction to variable and feature selection». En: *Journal of Machine Learning Research* (2003).

[45] Ilya Loshchilov y Frank Hutter. «Decoupled Weight Decay Regularization». En: *International Conference on Learning Representations*. 2019.

[46] Peter J Huber. «Robust estimation of a location parameter». En: *The Annals of Mathematical Statistics* (1964).

[47] Sergey Ioffe y Christian Szegedy. «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: *International Conference on Machine Learning, PMLR*. 2015.

[48] Nitish Srivastava et al. «Dropout: a simple way to prevent neural networks from overfitting». En: *The Journal of Machine Learning Research* (2014).

[49] Mark Treveil et al. *Introducing MLOps: How to scale machine learning in the Enterprise*. O'Reilly Media, 2020.

[50] Micha Gorelick y Ian Ozsvold. *High Performance Python: Practical Performant Programming for Humans*. O'Reilly Media, 2020.

```

1  {
2    "success": true,
3    "data": {
4      "key": {
5        "value": {
6          "key": "B", "mode": 0, "key_mode": "B Minor", "prob": "70.51%"
7        },
8        "key": "D", "mode": 1, "key_mode": "D Major", "prob": "17.52%"
9      },
10     "key": "E", "mode": 0, "key_mode": "E Minor", "prob": "10.32%"
11   },
12   "loudness": { "value": -7.966 },
13   "Fivep": {
14     "value": {
15       "acousticness": 0.622, "danceability": 0.502, "energy":
16       0.638, "instrumentalness": 0.555, "valence": 0.368
17     },
18     "Liveness": { "value": 0.116 },
19     "Speechiness": { "value": 0.035 },
20     "MoodPrimary": {
21       "top_5":
22       "Yearning 18.20%, Stirring 13.95%, Melancholy 9.87%, Brooding:
23       8.87%, Rowdy:7.67%"
24     },
25     "MoodSecondary": {
26       "top_5":
27       "Cool Melancholy 6.30%, Lyrical Sentimental 4.89%, Awakening /
28       Stately 4.64%, Light Melancholy 4.55%, Gentle Bittersweet 4.17%"
29     },
30     "GenreTarget": {
31       "top_5":
32       "Mainstream 90s and 00s Rock, Alternative Rock, General Latin
33       Rock, General Downtempo, General Indie Rock"
34     }
35   }
36 }

```

CÓDIGO C.1. Respuesta JSON de inferencia.

Bibliografía

- [1] J. Stephen Downie. «Music Information Retrieval». En: *Annual Review of Information Science and Technology* (2003).
- [2] Michael A. Casey et al. «Content-Based Music Information Retrieval: Current Directions and Future Challenges». En: *Proceedings of the IEEE* (2008).
- [3] Gordon C Bruner II. «Music, mood, and marketing». En: *Journal of marketing* (1990).
- [4] Franck V Garlin y Keri Owen. «Setting the tone with the tune: A meta-analytic review of the effects of background music in retail settings». En: *Journal of Business Research* (2006).
- [5] Holger Roschk, Sandra Maria Correia Loureiro y Jan Breitsohl. «Calibrating 30 years of experimental research: a meta-analysis of the atmospheric effects of music, scent, and color». En: *Journal of Retailing* (2017).
- [6] Plusyc Live SAS. *Sitio web oficial*. 2026. URL: <https://www.plusyc.com>.
- [7] Brian McFee et al. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference*. Visitado el 2026-03-20. 2015. URL: <https://librosa.org/>.
- [8] Dmitry Bogdanov et al. «Essentia: An Audio Analysis Library for Music Information Retrieval». En: *Proceedings of the 14th International Society for Music Information Retrieval Conference (ISMIR)*. 2013. URL: <http://essentia.upf.edu/>.
- [9] Jordi Pons y Xavier Serra. «musicnn: Pre-trained Convolutional Neural Networks for Music Audio Tagging». En: *Late-Breaking/Demo Session of the 20th International Society for Music Information Retrieval Conference (ISMIR)*. 2019. URL: <https://github.com/jordipons/musicnn>.
- [10] Jort F. Gemmeke et al. «Audio Set: An ontology and human-labeled dataset for audio events». En: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2017. URL: <https://research.google.com/audioset/>.
- [11] Spotify AB. *Spotify Web API Reference: Get Audio Features*. 2026. URL: <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>.
- [12] Gracenote Inc. *Gracenote Music Data: Sonic Attributes and Mood*. 2026. URL: <https://gracenote.com/products/audio-data/>.
- [13] Tunebat. *Tunebat: Song Key, BPM and Audio Features Database*. 2026. URL: <https://tunebat.com/>.
- [14] George Tzanetakis y Perry Cook. «Musical genre classification of audio signals». En: *IEEE Transactions on Speech and Audio Processing* (2002).
- [15] Michaël Defferrard et al. «FMA: A Dataset for Music Analysis». En: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017.
- [16] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.
- [17] Alexander Lerch. *An introduction to audio content analysis: Applications in signal processing and music informatics*. John Wiley & Sons, 2012.

- [18] Emilia Gómez. «Tonal description of polyphonic audio for music content processing». En: *INFORMS Journal on Computing* (2006).
- [19] Christopher Harte, Mark Sandler y Martin Gasser. «Detecting harmonic change in musical audio». En: *Proceedings of the 1st ACM workshop on Audio and music computing multimedia*. 2006.
- [20] Qiuqiang Kong et al. «PANNS: Large-scale pretrained audio neural networks for audio pattern recognition». En: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2020).
- [21] Karen Simonyan y Andrew Zisserman. «Very deep convolutional networks for large-scale image recognition». En: *International Conference on Learning Representations*. 2015.
- [22] Guolin Ke et al. «Lightgbm: A highly efficient gradient boosting decision tree». En: *Advances in neural information processing systems*. Vol. 30. 2017.
- [23] Takuya Akiba et al. «Optuna: A next-generation hyperparameter optimization framework». En: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019.
- [24] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep learning*. MIT press, 2016.
- [25] Sebastián Ramírez. *FastAPI*. 2018. URL: <https://github.com/fastapi/fastapi>.
- [26] Python Software Foundation. *Python Language Reference*. 2026. URL: <https://www.python.org/>.
- [27] Dirk Merkel. «Docker: lightweight linux containers for consistent development and deployment». En: *Linux journal* (2014).
- [28] Oracle Corporation. *Oracle Cloud Infrastructure*. 2026. URL: <https://www.oracle.com/cloud/>.
- [29] Thierry Bertin-Mahieux et al. «The million song dataset». En: *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR 2011)*. 2011.
- [30] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* (1998).
- [31] Bob L. Sturm. «The state of the art ten years after a state of the art: Future research in music information retrieval». En: *Journal of New Music Research* (2014).
- [32] Geoffroy Peeters. *A large set of audio features for sound description (similarity and classification) in the CUIDADO project*. Inf. téc. IRCAM (Institut de Recherche et Coordination Acoustique/Musique), 2004.
- [33] Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. 2020. URL: <https://www.kaggle.com/datasets/andradolteanu/gtzan-dataset-music-genre-classification>.
- [34] Shawn Hershey et al. «CNN architectures for large-scale audio classification». En: *IEEE*. 2017.
- [35] ITU-R. «Algorithms to measure audio programme loudness and true-peak audio level». En: *Recommendation ITU-R BS.1770-4* (2015).
- [36] Alain De Cheveigné e Hideki Kawahara. «YIN, a fundamental frequency estimator for speech and music». En: *The Journal of the Acoustical Society of America* (2002).
- [37] Earl Vickers. «The loudness war: Background, speculation and recommendations». En: *Audio Engineering Society*. 2010.

- [38] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.
- [39] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [40] Fabian Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* (2011).
- [41] Max Kuhn y Kjell Johnson. *Applied Predictive Modeling*. Springer, 2013.
- [42] Ron Kohavi. «A study of cross-validation and bootstrap for accuracy estimation and model selection». En: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*. Morgan Kaufmann Publishers Inc. 1995.
- [43] Leo Breiman et al. *Classification and Regression Trees*. CRC press, 1984.
- [44] Isabelle Guyon y André Elisseeff. «An introduction to variable and feature selection». En: *Journal of Machine Learning Research* (2003).
- [45] Ilya Loshchilov y Frank Hutter. «Decoupled Weight Decay Regularization». En: *International Conference on Learning Representations*. 2019.
- [46] Peter J Huber. «Robust estimation of a location parameter». En: *The Annals of Mathematical Statistics* (1964).
- [47] Sergey Ioffe y Christian Szegedy. «Batch normalization: Accelerating deep network training by reducing internal covariate shift». En: *International Conference on Machine Learning*. PMLR. 2015.
- [48] Nitish Srivastava et al. «Dropout: a simple way to prevent neural networks from overfitting». En: *The Journal of Machine Learning Research* (2014).
- [49] Mark Treveil et al. *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media, 2020.
- [50] Micha Gorelick y Ian Ozsvald. *High Performance Python: Practical Performant Programming for Humans*. O'Reilly Media, 2020.